

F2Z

immediate

August 20, 2026

Abstract

Albert: Should the name be F2F?

Contents

1 Introduction

1.1 Contributions

F2Z: Proving linear claims over any field (possibly an extension field) or integers, for a vector of bits committed on a binary field. [Albert: What happens if we replace FRI by F2Z in Plonky3?](#)

Follow up on Zinc+: we adopt its constraint system and greatly boost its performance with the aforementioned F2Z

Results in proof system that works with highly expressive and “small” constraints, and is competitive or outperforms other systems on many tasks of interests. Removes all the complexity of arithmetization, and in exchange requires you to work on a gnarly grand product.

New optimizations for grand products in binary fields, arising from our “in the exponent claims” (Binius64 already works with such grand products – we provide new optimizations).

Virtualization of XOR (or, more generally, $\mathbb{F}_2$ -linear transformations) of our committed witness elements, without the need to arithmetize on binary fields whatsoever.

Generalization/improvements to ring-switch in binary fields?

Benches: F2Z works with a highly expressible constraint system and outperforms in all axis on all measured tasks except in proving big quantities of hashes. For that task, prover time is 2-3x slower, while proof sizes are about 20-33% smaller, and memory usage is smaller too [Albert: how about the verifier?](#). For small number of hashes, say $<2^{10}$, F2Z outperforms on all axis [Albert: or so it seems](#). This may be because, with F2Z arithmetization, witnesses are smaller, and so floor costs in F2Z may be smaller than in other schemes.

1.2 Ramblings (warm-up for the true intro)

Why does this paper exist? F2Z has many similarities to Binius64, so why F2Z at all? First of all, Binius64 is an end-to-end proving system, while F2Z is a PCS. Let’s consider F2Z-SNARK a

SNARK for proving R1CS constraints over $\mathbb{Z}$ or $\mathbb{Z}[X]$ with ideals. Flock is also related to F2Z-SNARK, though less so, since Flock is a more use-case specific SNARK for proving bit-intensive computations. We should also keep in mind that Flock is, at the end of the day, Binius64 on steroid optimizations. In general, I think of Flock as subsumed by a Binius64 repo that adopts Flock's optimizations. Hence I will not discuss Flock too much here.

In what follows I understand F2Z-SNARK to be a proof system for constraints over $\mathbb{Z}, \mathbb{Z}[X]$ with ideals, binary fields, etc., like Zinc+ at the arithmetization and PIOP level (applied to R1CS-like constraints), but using F2Z as PCS.

Efficiency and costs I will now compare F2Z-SNARK with Binius64 on a operation-by-operation case:

- *Integer and modular addition.* F2Z-SNARK can handle addition or modular addition) of arbitrarily sized integers. Even if the integers are gigantic, the projection to random field trick compresses their size to 100-ish bits.

On the other hand, integer addition or modular addition (how does Binius handle modular addition when the modulus is not 2^n actually?) in Binius64 requires committing to the carry bits (so each single addition adds a new 32-64 bit word to the witness), and cannot handle large integer/modular addition without chunking.

Further, addition in Binius64 produces a non-linear constraint, because proving that the committed carry bits are indeed carry bits requires a Hadamard product. In F2Z-SNARK, additions are linear constraints.

- *Integer or modular multiplication.* This is a quadratic constraint in F2Z-SNARK. It does not introduce extra witness¹. Again, it supports arbitrary sizes and benefits from the random projection for large sizes.

Binius64 supports multiplication of elements of less than 32 bits. Larger sizes require chunking. Under 32 bits, it does not pay much for multiplication at the constraint level. The way it proves the multiplications is via a grand product similar to F2Z-SNARK grand product in MLE evaluations.

A computation where we are just making m multiplications of under 32 bit integers may actually end up costing almost the same between Binius64 and F2Z-SNARK. [Albert: Let's run the experiment](#)

It is interesting that in Binius64, multiplications don't go through the Spartan-like PIOP, and instead are proved directly with the grand product [Albert: At least that's my understanding](#). In F2Z-SNARK, multiplications would indeed go through the normal PIOP pipeline. Let's keep in mind as something F2Z-SNARK could mimic. [Albert: Interesting](#)

- *XOR*
- *AND*
- *Proof system costs*

¹Or it negligible does, because of the possible $+n \cdot w$ term in a modular equation, where n is the modulus and w is a committed integer –very small in bit size in practice

Specific performance This is WIP, but when it comes to hash proving, F2Z-SNARK seems to outperform in all axes Binius/Flock when there are less than 2^8 hashes to prove. Beyond that, the F2Z-SNARK route points to 1.5-3x prover times, comparable proof sizes, significantly smaller RAM usage, and unknown verifier comparison for now [Albert: WIP](#). The practical max number of hashes that F2Z-SNARK can handle is higher than in Flock due to the lesser memory usage.

The fact that F2Z-SNARK performs better for smaller number of hashes may be due to F2Z-SNARK working with a smaller witness. At those sizes, there's likely floor costs (e.g. Merkle openings) that somewhat dominate costs, and having a 2x (roughly) smaller witness may start to be relevant.

Interesting to note that in many use cases (e.g. digital Id) one has less than 2^8 hashes to prove. Flock-like numbers come up in Ethereum signature aggregation and maybe proof of provenance where one has to prove a hash of a large file (and others?).

[Albert: Current WIP: hash + ECDSA sign comparison. Quantized ML linear layer](#)

For initial reference, Binius64 takes about 110ms to prove an ECDSA with a 200KB proof size.

Scientific contributions So far, we have compared Binius64 and F2Z-SNARK in terms of performance. But how about the scientific contributions of F2Z-SNARK?

F2Z-SNARK is a proof system designed to handle constraints over $\mathbb{Z}$, $\mathbb{Z}[X]$ with ideal membership, binary fields, and other fields, simultaneously (in v3 we are probably sticking to integers and $\mathbb{Z}[X]$ for simplicity). Arithmetization and PIOP-wise, it borrows from Zinc+, and the novelty compared to Zinc+ is that we are committing to the integer bits over a binary field. For bit-intensive computations, this is much better than committing over $\mathbb{Z}$. It remains to be seen what happens in integer-heavy computations [Albert: TODO! Edit: John Wu already compared it to Hyrax](#).

Potentially, we could commit to large integers with a different PCS over $\mathbb{Z}$, say with IPRS, and create a recursive proof of opening for the two PCS's, using F2Z since PCS opening proofs are hash heavy.

Thus some scientific contributions of F2Z-SNARK are

- Unlocks good scaling of bit intensive computations in Zinc+-like proof systems.
- If we include Zinc+ in this line of work, F2Z-SNARK is a follow up for a proof system that uses a highly expressive constraint system. Due to the expressivity, the use-cases of F2Z-SNARK are potentially huge, and difficult to predict. For example, the F2Z-PCS could be used as a drop-in replacement for elliptic curve PCS's (as shown by John Wu)

We (tentatively) choose F2Z-SNARK (likely with both WHIZ and F2Z) working exclusively on $\mathbb{Z}$ or $\mathbb{Z}[X]$ as our route for a general zkDSL, and the motivation was the expressivity of the constraints.

- F2Z-SNARK does not quite provide much novelty regarding SNARKs over binary fields, beyond possibly some optimizations in the grand product proof (e.g. lower levels of the grand product GKR are computed via a lookup table, exploiting the limited options we have in leaves due to the bit-ness of the witness).

That being said, some of these optimizations could be published as contributions, following Flock's lead. [Albert: Sometimes I undersell my work, this is a memo to myself to be mindful with that](#)

	F2Z-PCS	Zinc+-PIOP	Binius64	Flock	Plonky3	Zinc+
ECDSA			119/6.3/195/?	NA		
RSA				NA		
2^{17} int mul 16bits	48/2/80/62		217/9/313/?	NA	367/7/340/?	
2^{17} int mul 32bits	74/2.4/86/124		218/9/313/?	NA	571/7.6/373/?	
1M int mul 16bits	192/2.9/112/483		3600/61/440/?	NA	2920/9.8/492/?	
1M int mul 32bits	266/3.7/119/863		3800/62/440/?	NA	5050/10.2/520/?	
2^{17} BabyBear mul	56/2.7/142/88			NA	58/6.3/307/99	
2^{20} BabyBear mul	328/4.1/216/682			NA	411/10.4/473/713	
2^7 SHA	12.7/1.75/71/14*	expected negl.	37/3.4/249/?	33/25/112/?		
2^{11} SHA	93.5/2.8/109/197*	expected negl.	126/27/381/?	47.2/27/172/?		
2^{15} SHA	1320/4.8/170/3080*	expected negl.	Stuck	425/25/233/?		
2^8 Blake3	25/2.2/100/23	expected negl.	31/2.6/249/?	256/254/111/?	28/78/4524/48	
2^{11} Blake3	110/3.5/151/174	expected negl.		253/249/153/?	155/86/4582/179	
2^{15} Blake3	1257/5.4/322/2696	expected negl.	Stuck	386/267/160/?	2696/94/4717/2477	

Table 1: *= we are not using virtualization of XOR. The committed and opened witness size is 4x larger than it could be. [Albert](#): Missing costs for F2Z PIOP. Note these are negligible for the hash benches though, and likely quite less than the grand product cost in other benches. All these were obtained asking AI to run benches. For the final paper we should make sure we are doing fair and correct benching (e.g. I’m not sure memory consumption is measured the same way across benches, especially for Binius, which is max RSS), but this helps get a rough idea how F2Z performs relatively to other systems. Prover_time(ms)/Verifier_time(ms)/Proof_size(KB)/Peak_memory(MB). Integer mult measure one million u16 x u16 -> u32 mults. F2Z-PCS does not include PIOP costs – these are negligible in the hash benches (linear constraints), but should be accounted for in the other benches (degree 2 constraints). Ran in MacBook Air M4 16GB.

Flock’s verifier seems a bit broken in general. Flock’s Blake3 prover for small number of hashes seems broken too. I modified Flock’s Ligerito config so that it produces smaller proofs at very little extra cost. For Blake3 we couldn’t do 2^7 hashes because Flock errors out on 2^7 hashes [Albert](#): [todo: make SHA also do \$2^8\$ instead of \$2^7\$](#)

F2Z-PCS hash performance was “proxied” as follows: for SHA-256, 2^{14+k} witness corresponds to 2^k hashes. Better arithmetization may be possible. We *DID NOT* leverage virtualization of XOR, which allows to run Ligerito for 2^{12+k} witness size, rather than 2^{14+k} (The GKR grand product still would use 2^{14+k} factors). [Albert](#): [VERY IMPORTANT: It is possible that the GKR grand product can be optimized in this setting. Right now we are just proving a \$2^{14+k}\$ on unstructured bits, but in reality many of these bits are linear combinations in \$\mathbb{F}_2\$ of witness bits.](#)

Binius64 achieves the highest number of mults per second at 2^{17} mults. Binius64 at multiplication of 16bit integers pays a penalty because the system uses 64-bit words by default, so it either uses a 64 bits per 16bit witness element and range check the word, or pack different 16 and 32 bit integers into the same 64 bit word, but then it has to relate the product of the 64 bit words to the products of the original words (or unpack them as 64 bit words before multiplying). OTOH doing more Binius-native mults (32x32 -> 64 bits) does not perform amazingly either [Albert](#): [This most likely heavily disproves my belief that Binius64 and F2Z-SNARK are similar in integer multiplication. This is a good showcase of the flexibility of arithmetizing the way we do: we don’t need to use predefined word sizes: here our R1CS is just reconstruction of integers via linear combinations with powers of twos, and Hadamard product \(note that we are not measuring the cost of the PIOP in F2Z! But it should be cheaper than the grand product, and amenable to some better optimization \(univariate skip e.g.\). But in any case, I’m not even sure the issue is the rigidity of the arithmetization in Binius64, since 32x32->64 is Binius friendly as far as I understand, and performance is mid](#)

BabyBear mul rows measure native BabyBear field mults (medians of 3). Plonky3 = native 3-column AIR ($a \cdot b = c$, no range checks), challenges in the quartic extension BabyBear⁴, TwoAdicFriPcs with FriParameters::new_benchmark (rate 1/2, 100 queries, 16-bit query PoW), Poseidon2 Merkle; a complete STARK (includes the quotient commitment). F2Z-PCS = the trace’s bit decomposition ($3 \times 31 = 93$ bits/mul, zero-padded to $2^{24} / 2^{27}$ cells) committed over $\mathbb{F}_2$ and opened at a point in the same quartic extension $\mathbb{K} = \mathbb{F}_p[X]/(X^4 - 11)$ via Step 3 of ?? (the random-prime projection, $q' \approx 100$ bits); prover = commit + open (3.3 + 52.3 / 23.7 + 303.8 ms); PIOP (zerocheck) costs excluded, as for the other F2Z-PCS rows; ± 10 –15% run-to-run band on prove medians. Peak memory: Plonky3 measured as max RSS, F2Z as heap high-water – comparable but not identical notions.

Blake3 rows, Plonky3 column: Blake3Air (9168 columns, one compression per row; 8704 boolean + 464 sixteen-bit-limb columns $\approx 2^{14}$ committed bits per compression) over BabyBear at the same new_benchmark FRI settings – the 100 query openings of the 9168-wide trace dominate, hence the ≈ 4.5 –4.7 MB proofs and ≈ 80 –95 ms verify. The F2Z-PCS Blake3 cells were re-measured in the same session/box as commit + open of the same trace’s bits (2^{14+k} cells for 2^k compressions) at a BabyBear⁴ point via Step 3 of ?? (prover = commit + open: 2.0 + 23.1 / 5.6 + 103.9 / 123.1 + 1134.3 ms), superseding the earlier prime-field-point measurements.

There's also the virtualization of XOR elements of F2Z-PCS. In a sense, a similar thing occurs in Binius64: their R1CS constraint system admits hadamard products, replacing the hadamard by integer multiplication. So such constraint is multiplying together linear combinations of the witness (over the binar field, so "XOR combinations").

Our virtualization trick can be understood as if we had an R1CS over $\mathbb{Z}$, with witness $\mathbf{z}$, and an extra constraint $\mathbf{z} = \pi_2^{-1}(M\pi_2(\mathbf{z}))$ for some matrix M . What Binius is doing can be understood similarly, but with much more limited expressivity. Hence in this sense Binius64 arithmetization is a specific case of our arithmetization.

Albert: Noting so I don't forget: the idea of F2Z being a drop-in replacement alternative to Hyrax (PQ secure and more efficient) may be a good extra motivating point

1.3 Acknowledgements

2 Preliminaries

General notation Given an integer $n \geq 1$, we write $[n]$ to denote the set $\{1, \dots, n\}$. Any map $f : S \rightarrow T$ is extended entrywise to vectors and matrices with entries in S , and coefficient-wise to polynomials with coefficients in S .

2.1 Algebra

We write $\mathbb{F}_q$ for the finite field with q elements (where q is a prime or a prime power), $\mathbb{Q}$ for the field of rational numbers, and $\mathbb{Z}$ for the ring of integers.

The *characteristic* of a ring R is the largest $c \geq 1$, if it exists, such that $1 + \dots + 1 = 0$ in R . Otherwise, if $1 + \dots + 1 \neq 0$ for all c , the characteristic is zero. By $[0..n]_R$ we denote all elements in R of the form $1 + \dots + 1$ for $i = 0, \dots, n$. For us, typically $R = \mathbb{Z}$ or R is a finite field $\mathbb{F}$.

We let $R[X]$ be the ring of univariate polynomials with coefficients in R , on the variable X . For a tuple of variables $\mathbf{Y} = (Y_1, \dots, Y_\mu)$, $R[\mathbf{Y}]$ denotes multivariate polynomials with coefficients in R . Given a degree bound $d \geq 1$, a bit-size bound $B \geq 1$, we define

$$R_{\leq B}^{\leq d}[X] := \left\{ a(X) \in R[X] \mid a(X) = \sum_{i=0}^d a_i \cdot X^i, \text{ for some } a_i \in [0..2^B - 1]_R, \text{ for all } i \right\},$$

When we place no degree or bit-size bound, we use the notation $R_{\leq B}[X]$ and $R^{\leq d}[X]$, respectively. Note that $R^{\leq d}[X]$ is an R -module of dimension $(d+1)$, admitting as base $1, X, \dots, X^d$.

Given a ring homomorphism $\psi : R_1 \rightarrow R_2$, a R_1 -module M_{R_1} , and a R_2 -module, a ψ -module homomorphism between M_{R_1} and M_{R_2} is a map ϕ such that $\phi(\alpha \cdot a + \beta \cdot b) = \psi(\alpha) \cdot \phi(a) + \psi(\beta) \cdot \phi(b)$ for all $\alpha, \beta \in R_1$ and $a, b \in M_{R_1}$.

Given a base $\alpha_1, \dots, \alpha_d$ for M_R , we let $M_{R_{\leq B}}$ be the subset of M_R with elements whose coordinates belong to $R_{\leq B}$ in this base.

Definition 2.1 (Projections to finite fields). *Albert: Maybe place somewhere else since it's a non-standard definition* For a prime q , we let $\pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q$ be the natural reduction modulo q homomorphism. For convenience, we also let

$$\pi_q : \mathbb{F}_{q^e} \rightarrow \mathbb{F}_{q^e}$$

denote the identity homomorphism on any field of characteristic q $\mathbb{F}_{q^e}$.

Let $\mathbb{K}$ be a finite field of characteristic q , let $\mathbb{K}'$ be an extension of $\mathbb{K}$ (possibly $\mathbb{K}' = \mathbb{K}$) and let $\alpha \in \mathbb{K}'$. By $\pi_{q,\alpha}$ we denote the following maps:

$$\begin{aligned} \pi_{q,\alpha} : \mathbb{Z}[X] &\rightarrow \mathbb{K}' & \pi_{q,\alpha} : \mathbb{K}[X] &\rightarrow \mathbb{K}' \\ h(X) &\mapsto \pi_q(h)(\alpha) & h(X) &\mapsto h(\alpha) \end{aligned}$$

A subset $S \subseteq R$ of a commutative ring R is called *exceptional* if for all two distinct $a, b \in R$, the element $a - b$ has a multiplicative inverse in R .

Lemma 2.2 (Generalized Alon–Füredi Theorem [BCPS18]). *Let R be a ring and let $S \subseteq R$ be a finite exceptional subset of R . Let $f(\mathbf{Y}) \in R[\mathbf{Y}]$ be a nonzero multilinear polynomial in μ variables. Then $\Pr_{\mathbf{r} \leftarrow S^\mu} [f(\mathbf{r}) = 0] \leq \mu/|S|$.*

2.2 Linear codes and relations

Linear codes Let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code over $\mathbb{F}$ of dimension ℓ and blocklength m . For $h \geq 1$, the h -interleaving of $\mathcal{C}$ is the code $\mathcal{C}^{\equiv h} \subseteq (\mathbb{F}^h)^m$ of dimension $\ell \cdot h$ and blocklength $m \cdot h$, whose codewords have the form

$$((c_{i1}, \dots, c_{ih}))_{i \in [m]} \quad \text{for } (c_{1j}, \dots, c_{mj}) \in \mathcal{C} \quad \text{for all } j \in [h].$$

Relations A relation $\mathcal{R}$ consists of triples $(\mathbf{x}, \mathbf{y}; \mathbf{w})$, where $\mathbf{x}$ is the explicit instance, $\mathbf{y}$ the implicit instance or oracle, and $\mathbf{w}$ the witness. The corresponding language is $\text{LANG}(\mathcal{R}) := \{(\mathbf{x}, \mathbf{y}) \mid \exists \mathbf{w} \text{ such that } (\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}\}$.

Polynomial Equation Satisfiability (PESAT) over finite fields We recall a relation from [linear-accumulation] [Albert: Linear time accumulation paper, or earlier?](#) over finite fields, which captures popular constraints such as R1CS, CCS, AIR (cf. [linear accumulation]), and lookup and permutation constraints (cf. [zinc]).

Fix a finite field $\mathbb{F}$, a witness length ℓ , an input length k , a number of rows n , a collection of n polynomials $\mathbf{p} = (\mathbf{p}_1, \dots, \mathbf{p}_n)$ on $k + \ell$ variables, for each $j \in [n]$.

Definition 2.3. The PESAT relation with respect to these parameters is defined as

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}) = \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \mathbb{F}^k, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}^\ell \end{array} \middle| \mathbf{p}_j(\mathbf{x}, \mathbf{f}) = 0 \text{ for all } j \in [n] \right\}$$

Let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code of dimension ℓ' and blocklength m over $\mathbb{F}$. Let $T : \mathbb{F}^\ell \rightarrow \mathbb{F}^{\ell'}$ be an arbitrary map (looking ahead, for us T will be a packing map that packs elements from $\mathbb{F}_2$ into elements of a larger binary field). We define the *Encoded PESAT (Enc-PESAT) relation* as:

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}, \mathcal{C}, T) := \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \mathbb{F}^k, \\ \mathbf{y} = f \in \mathbb{F}^m, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}^\ell \end{array} \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(T(\mathbf{f})) = f \\ \wedge \mathbf{p}_j(\mathbf{x}, \mathbf{f}) = 0 \text{ for all } j \in [n] \end{array} \right\}$$

When $2^\nu = 1$ and $\mathbf{x}$ are such that $\mathbf{x} = (\mathbf{v}, \mu) \in \mathbb{F}^\ell \times \mathbb{F}$; $\mathbf{p}_1(\mathbf{x}, \mathbf{f}) = \langle \mathbf{f}, \mathbf{v} \rangle - \mu$; and $T = \text{Id}$ (and $\ell' = \ell = k$), the relation $\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}, \mathcal{C}, \text{Id})$ is called a *constrained code* [WHIR?] [Albert: WHIR?](#). We denote it

$$\mathcal{R}_{\text{Lin}}(\mathbb{F}, \mathcal{C}, T).$$

2.3 Interactive Oracle Reductions (IOR) of proximity

[Albert: We don't use IOPPs in the paper? Make sure we don't mention them later](#)

A public-coin *Interactive Oracle Reduction (IOR)* of proximity $\Pi = (\mathbf{P}, \mathbf{V})$ from a relation $\mathcal{R}$ to a relation $\mathcal{R}'$ is specified as follows. The prover $\mathbf{P}$ receives $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$. The verifier $\mathbf{V}$ receives $\mathbf{x}$ and query access to $\mathbf{y}$. They interact for k rounds. In round $j \in [k]$, the prover sends an oracle π_j and the verifier sends a uniformly random message $\rho_j \in \{0, 1\}^{r_j}$. After the k rounds, the verifier makes queries to $\mathbf{y}$ and to $\pi_1, \dots, \pi_k$. Then:

- $\mathbf{V}$ either rejects or outputs a new pair $(\mathbf{x}', \mathbf{y}')$, where $\mathbf{y}'$ is specified implicitly in terms of the oracles $\mathbf{y}$ and $\pi_1, \dots, \pi_k$.
- $\mathbf{P}$ outputs a witness $\mathbf{w}'$ such that $(\mathbf{x}', \mathbf{y}', \mathbf{w}') \in \mathcal{R}'$.

We write $\langle \mathbf{P}, \mathbf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w})$ for the interaction of prover and verifier during the IOR.

Definition 2.4 (Completeness). Π has (*perfect*) *completeness* if for every $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$,

$$\Pr_{\rho_1, \dots, \rho_k} \left[(\mathbf{x}', \mathbf{y}'; \mathbf{w}') \in \mathcal{R}' \mid (\mathbf{x}', \mathbf{y}'; \mathbf{w}') \leftarrow \langle \mathbf{P}, \mathbf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w}) \right] = 1.$$

Remark 2.5. An IOP for a relation $\mathcal{R}$ is an IOR from $\mathcal{R}$ to the trivial relation $\mathcal{R}_{\text{triv}} = \{(\mathbf{x} = \perp, \mathbf{y} = \perp, \mathbf{w} = \perp)\}$.

We proceed to define round-by-round knowledge soundness of an IOR, as presented in [TCC:BCFW25].

Definition 2.6 (Knowledge state function). Let $\Pi = (\mathbf{P}, \mathbf{V})$ be an IOR from a relation $\mathcal{R}$ to $\mathcal{R}'$. A *knowledge state function* for Π is a (possibly inefficient) function KState that, on input $(\mathbf{x}, \mathbf{y})$, interaction transcript tr , and knowledge state witness $\mathbf{w}$, outputs a bit and has the following properties.

- **Empty transcript.** If $\text{tr} = \emptyset$ is the empty transcript, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}; \mathbf{w}) \in \mathcal{R}$.
- **Prover moves.** If tr is a transcript where the prover is about to move and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \pi, \mathbf{w}) = 0$ for every prover message π .
- **Full transcript.** If $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $\mathbf{V}$ on tr outputs $(\mathbf{x}', \mathbf{y}')$ such that $(\mathbf{x}', \mathbf{y}'; \mathbf{w}) \in \mathcal{R}'$.

Definition 2.7 (Round-by-round knowledge soundness). A k -round IOR Π has *round-by-round knowledge soundness* with errors $(\kappa_1, \dots, \kappa_k)$ if there exists a knowledge state function KState as in ?? and a deterministic extractor $\mathcal{E}_{\text{rbr}}$ such that for every $(\mathbf{x}, \mathbf{y})$, every $j \in [k]$, and every transcript tr where the verifier is about to send ρ_j ,

$$\Pr_{\rho_j} \left[\exists \mathbf{w} \mid \begin{array}{l} \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_j, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_j, \mathbf{w}) = 1 \end{array} \right] \leq \kappa_j(\mathbf{i}, \mathbf{x}).$$

Remark 2.8 (Composition of IORs). [Albert: What can I cite for this?](#) IORs can be composed together and inherit completeness and round-by-round knowledge soundness errors. More precisely,

let Π_1, Π_2 be perfectly complete IORs from $\mathcal{R}_1$ to $\mathcal{R}_2$ and $\mathcal{R}_3$, respectively, with round-by-round knowledge errors $\kappa_{11}, \dots, \kappa_{1k_1}$ and $\kappa_{21}, \dots, \kappa_{2k_2}$. Consider the IOR Π' from $\mathcal{R}_1$ to $\mathcal{R}_3$ consisting of applying IOR Π_1 first, and then Π_2 taking as input for Π_2 the output of Π_1 . Then Π' is a perfectly complete $(k_1 + k_2)$ -round IOR from $\mathcal{R}_1$ to $\mathcal{R}_3$ with round-by-round knowledge soundness errors $\kappa_{11}, \dots, \kappa_{1k_1}, \kappa_{21}, \dots, \kappa_{2k_2}$.

Albert: Describe **KState** for the composed IOR?

3 $\mathbb{F}_2\mathbb{Z}$ IOPP for Lin-ModFieldBin

3.1 The Lin-ModFieldBin and the Lin-BitsDual relations

Let R denote either $\mathbb{Z}$ or a finite field $\mathbb{F}$ of characteristic q ; let M_R be an R -module of finite dimension $d \geq 1$; let $\mathbb{K}$ be a finite field of characteristic q (equal or not to $\mathbb{F}$); and let $\mathbb{F}_{2^n}$ be a binary field. Intuitively, our witness entries belong to M_R ; $\mathbb{K}$ is the field where our constraints on (a projection of) the witness live; and $\mathbb{F}_{2^n}$ is the field over which our witness is committed to (through its bits).

For example, M_R may be simply R , i.e. $\mathbb{Z}$ or $\mathbb{F}$; or it may be $R^{<d}[X]$, i.e. the subset of univariate polynomials of degree $< d$ with coefficients in R .

Let

$$\psi_M : M_R \rightarrow \mathbb{K}$$

be a π_q -module homomorphism **Albert: correct terminology?**, where, recall, $\pi_q : R \rightarrow \mathbb{K}$ is reduction modulo q if $R = \mathbb{Z}$, or the identity map if R is a field of characteristic q . For example, if $M_R = R^{<d}[X]$, ψ could be one of the maps $\pi_{q,\alpha}$ from ?? (reduction modulo q if $\mathbb{Z} = R$ and evaluation at $X = \alpha \in \mathbb{K}$, with $\mathbb{K}$ having characteristic q).

Let $\gamma_1, \dots, \gamma_d$ be a base for M_R .

Packing in $\mathbb{F}_{2^n}$. Let $\beta_1, \dots, \beta_n \in \mathbb{F}_{2^n}$ be an $\mathbb{F}_2$ -base of $\mathbb{F}_{2^n}$ seen as an $\mathbb{F}_2$ -vector space of dimension n . Given a vector $\mathbf{u} \in \mathbb{F}_2^m$, with 2^n dividing m , we define

$$\text{Pack}_n(\mathbf{u}) \in \mathbb{F}_{2^n}^{m/n}$$

as the vector whose j -th entry is

$$\text{Pack}_n(\mathbf{u})_j = \mathbf{u}_{(j-1) \cdot n + 1} \cdot \beta_1 + \dots + \mathbf{u}_{(j-1) \cdot n + n} \cdot \beta_n \in \mathbb{F}_{2^n}.$$

The bits of a vector over $M_{R \leq B}$. Fix a vector length ℓ_0 . Let $\mathbf{f} \in M_{R \leq B}^{\ell_0}$ and write $\mathbf{f} = \sum_{j \in [d]} \mathbf{f}_{*j} \cdot \gamma_j$ with $\mathbf{f}_{*j} \in R_{\leq B}^{\ell_0}$ for all $j \in [d]$. Then write

$$\mathbf{f}_{*j} = \sum_{k=1}^B \mathbf{f}_{*jk} \cdot 2^{k-1}$$

for some vectors

$$\mathbf{f}_{*jk} \in \{0, 1\}^{\ell_0} \subseteq R^{\ell_0},$$

for all j, k . We define $\text{bits}(\mathbf{f}) \in \{0, 1\}^{\ell_0 \cdot d \cdot B} \subseteq \mathbb{Z}^{\ell_0 \cdot d \cdot B}$ as the vector with integer entries obtained by concatenating the vectors $\mathbf{f}_{*jk}$ and, if $R = \mathbb{F}$, lifting their entries to $\{0, 1\} \subseteq \mathbb{Z}$ via π_q^{-1} , i.e.

$$\text{bits}(\mathbf{f}) := (\text{lift}(\mathbf{f}_{*jk}) \mid j \in [d], k \in [B]) \in \{0, 1\}^{\ell_0 \cdot d \cdot B} \subseteq \mathbb{Z}^{\ell_0 \cdot d \cdot B},$$

where lift is the identity function if $R = \mathbb{Z}$, and $\text{lift} = \pi_q^{-1}$ if $R = \mathbb{F}$.

The Lin-ModFieldBin relation Let $\mathcal{C}$ be a linear code over $\mathbb{F}_{2^n}$ of dimension $(\ell_0 \cdot B \cdot d)/n$ and blocklength m .

The goal of this section is to provide an IOP of Proximity for the following relation: [Albert: should we add some parameters inside parenthesis as done with other relations?](#)

$$\mathcal{R}_{\text{Lin-ModFieldBin}} = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{v}, \mu) \in \mathbb{K}^{\ell_0} \times \mathbb{K}^d, \\ \mathbf{y} = f \in \mathbb{F}_{2^n}^m, \\ \mathbf{w} = \mathbf{f} \in M_{R \leq B}^{\ell_0} \end{array} \right) \middle| \begin{array}{l} f = \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\text{bits}(\mathbf{f})))), \\ \langle \psi_M(\mathbf{f}), \mathbf{v} \rangle = \mu \end{array} \right\}.$$

The Lin-BitsDual relation Our IOP for $\mathcal{R}_{\text{Lin-ModFieldBin}}$ is built from an IOP to what we call the *Linear constraint for bits over dual fields (Lin-BitsDual) relation*. This is essentially the same as $\mathcal{R}_{\text{Lin-ModFieldBin}}$, except that the witness $\mathbf{f}$ consists entirely of integer bits. Precisely, given a witness size parameter $\ell \geq 1$ and a linear code $\mathcal{C} \subseteq \mathbb{F}_{2^n}^m$ of dimension $\ell/2^n$ and blocklength m over $\mathbb{F}_{2^n}$, we define

$$\mathcal{R}_{\text{Lin-BitsDual}} := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{K} \times \mathbb{K}^{\ell}, \\ \mathbf{y} = f \in \mathbb{F}_{2^n}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^{\ell} \subseteq \mathbb{Z}^{\ell} \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f \\ \wedge \langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \mu \end{array} \right\}$$

3.2 Tensor decomposition, interleaved codes, and products of constrained codes

Let $(\mathbf{x}, \mathbf{y}, \mathbf{w}) = ((\mu, \mathbf{v}), f, \mathbf{f}) \in \mathcal{R}_{\text{Lin-BitsDual}}$. We fix a tensor decomposition of vectors $\mathbf{v} \in \mathbb{K}^{\ell}$ as

$$\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} = \left(\mathbf{v}_i^{(1)} \cdot \mathbf{v}_j^{(2)} \right)_{i \in [\ell_1], j \in [\ell_2]}$$

for some $\mathbf{v}^{(1)} \in \mathbb{K}^{\ell_1}, \mathbf{v}^{(2)} \in \mathbb{K}^{\ell_2}$, with $\ell_1 \cdot \ell_2 = \ell$. Note that the trivial tensor decomposition is admitted, i.e. we may effectively not decompose $\mathbf{v}$ by choosing $\ell_1 = \ell, \ell_2 = 1, \mathbf{v}^{(1)} = \mathbf{v}$, and $\mathbf{v}^{(2)} = (1)$.

We assume $\mathcal{C} = \mathcal{C}_1^{\equiv \ell_2}$ is the ℓ_2 -dimensional interleaving of a code $\mathcal{C}_1 \subseteq \mathbb{F}_{2^n}^{m/\ell_2}$ of dimension $\ell_1/2^n$ and blocklength m/ℓ_2 . We then index the entries of $\mathbf{f}$ as $\mathbf{f} = (\mathbf{f}_{ij})_{i \in [\ell_1], j \in [\ell_2]}$. Note that

$$\langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \sum_{j \in [\ell_2]} \left(\sum_{i \in [\ell_1]} \pi_q(\mathbf{f}_{ij}) \cdot \mathbf{v}_i^{(1)} \right) \cdot \mathbf{v}_j^{(2)} = \sum_{j \in [\ell_2]} \langle \pi_q(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle \cdot \mathbf{v}_j^{(2)},$$

where $\mathbf{f}_{*j} = (\mathbf{f}_{ij})_{i \in [\ell_1]}$.

Define the following bivariate polynomial:

$$P(Y, Z) = (Y - 1) \cdot Z + 1$$

Note $P(Y, 0) = 1$ and $P(Y, 1) = Y$. Consider the relation (cf. ??) $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}2\mathbb{Z}}, \mathcal{C}_1, \text{Pack}_n)$ where

$$Q_{\mathbb{F}2\mathbb{Z}}(Y_1, \dots, Y_{\ell_1+1}, Z_1, \dots, Z_{\ell_1}) := \prod_{i=1}^{\ell_1} P(Y_i, Z_i) - Y_{\ell_1+1},$$

that is,

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}2\mathbb{Z}}, \mathcal{C}_1, \text{Pack}_n) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{y}, z) \in \mathbb{F}_{2^n}^{\ell_1} \times \mathbb{F}_{2^n}, \\ \mathbf{y} = f \in \mathbb{F}_{2^n}^{m/\ell_2}, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}_{2^n}^{\ell_1} \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\mathbf{f})) = f \\ \wedge \prod_{i=1}^{\ell_1} ((\mathbf{y}_i - 1) \cdot \mathbf{f}_i + 1) = z \end{array} \right\}$$

We assume prover and verifier have access to an IOR Π_{red} that reduces $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}2\mathbb{Z}}, \mathcal{C}_1, \text{Pack}_n)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^n}, \mathcal{C}_1, \text{Pack}_n)$. In our instantiation Π_{red} is an optimized GKR protocol for grand products, which reduces the initial grand product claim to MLE evaluation claims of $\mathbf{y}$ and $\mathbf{f}$ over $\mathbb{F}_{2^n}$. As we explain in ??, we design several optimizations for the specific shape of our grand products.

We assume Π_{red} works so that the verifier never queries $\mathbf{y}$. Note that this is indeed the case with GKR (indeed, GKR reduces the initial circuit evaluation claim on the input and outputs, into MLE evaluation claims of the input and outputs, without querying any oracle in-between).

Further, we consider the ℓ_2 -fold parallel composition $\Pi_{\text{red}}^{\ell_2}$ of Π_{red} , and we assume that given ℓ_2 triples

$$\left((\mathbf{y}^{(j)}, \mu^{(j)}), g^{(j)}, \mathbf{h}^{(j)} \right)_{j \in [\ell_2]} \in (\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}2\mathbb{Z}}, \mathcal{C}_1, \text{Pack}_n))^{\ell_2}$$

the IOR $\Pi_{\text{red}}^{\ell_2}$ outputs

$$\left((\mathbf{v}, \mu_j) \in \mathbb{F}_{2^n}^{\ell_1} \times \mathbb{F}_{2^n}, g^{(j)}, \mathbf{h}^{(j)} \right)_{j \in [\ell_2]} \in (\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^n}, \mathcal{C}_1, \text{Pack}_n))^{\ell_2},$$

for some $\mathbf{r} \in \mathbb{F}_{2^n}^{\log \ell_1}$ and some $\mu_j \in \mathbb{F}_{2^n}$ for $j \in [\ell_2]$. In words, $\Pi_{\text{red}}^{\ell_2}$ outputs ℓ_2 linear claims for the input witnesses $\mathbf{h}^{(j)}$ behind the input oracles $g^{(j)}$, all *on the same vector of coefficients* $\mathbf{v}$. Again, in our instantiation $\Pi_{\text{red}}^{\ell_2}$ is simply the GKR protocol applied to several grand product claims at once, with optimizations discussed in ??.

3.3 The core $\mathbb{F}2\mathbb{Z}$ IOP for Lin-BitsDual

We provide an IOP for $\mathcal{R}_{\text{Lin-BitsDual}}$ next. We assume the notation from the previous two ?????. Given a projection $\pi_{q'} : \mathbb{Z} \rightarrow \mathbb{F}_{q'}$, we fix an inverse map of the form $\pi_{q'}^{-1} : \mathbb{F}_{q'} \rightarrow [0 \dots q' - 1] \subseteq \mathbb{Z}$. Let $\pi_{\text{canon}} : \mathbb{Z}[X] \rightarrow \mathbb{K}$ be the canonical homomorphism from $\mathbb{Z}[X]$ to $\mathbb{K}$. Namely, if $\mathbb{K} = \mathbb{Z}[X]/(q, h(X))$ for a polynomial $h(X)$ irreducible over $\mathbb{F}_q[X]$, then π_{canon} consists in modding q out, and taking the quotient by $(h(X))$ (if $\mathbb{K}$ is the prime field $\mathbb{F}_q$, then $\pi_{\text{canon}} = \pi_q$). We fix an inverse for π_{canon} of the form

$$\pi_{\text{canon}}^{-1} : \mathbb{K} \rightarrow [0 \dots q - 1]^{\leq \deg(\mathbb{K})}[X] \subseteq \mathbb{Z}.$$

In words, π_{canon}^{-1} simply takes an element from $\mathbb{K}$ represented as a degree $< \deg(h(X))$ polynomial over $\mathbb{F}_q$, and applies π_q^{-1} to it. We remark that one could choose any other such “inverse” maps, e.g. for $\pi_{q'}^{-1}$ one may prefer to use centered representatives.

Construction 3.1 (IOP for $\mathcal{R}_{\text{Lin-BitsDual}}$). The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{K} \times \mathbb{K}^\ell$ and $\mathbf{y} = f \in \mathbb{F}_{2^n}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}$. We assume the tensor split from the previous Section ??, and adopt its notation. Let $g \in \mathbb{F}_{2^n}$ be a nonzero element from $\mathbb{F}_{2^n}$ with sufficiently large multiplicative order, to be determined later.

1. Round 1: lifting the constraint over $\mathbb{K}$ to $\mathbb{Z}$

- The prover sends $\mu_1, \dots, \mu_{\ell_2} \in [0 \dots q - 1]^{\leq \deg(\mathbb{K})}[X]$ such that

$$\langle \mathbf{f}_{*j}, \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \rangle = \mu_j \quad \text{for all } j \in [\ell_2]. \quad (1)$$

- The verifier checks that

$$\sum_{j \in [\ell_2]} \pi_{\text{canon}}(\mu_j) \cdot \mathbf{v}_j^{(2)} = \mu \quad \text{over } \mathbb{K}. \quad (2)$$

Next, prover and verifier will focus on certifying that (??) holds for all $j \in [\ell_2]$.

In ?? we describe a variation in which, instead of sending the elements $\mu_1, \dots, \mu_{\ell_2}$, the prover commits to their bit-decomposition over $\mathbb{F}_{2^n}$, and proves (??) the present IOR. This saves on proof size and verifier time when ℓ is large.

- If $\deg(\mathbb{K}) = 0$, continue to the next step. Otherwise, if $\deg(\mathbb{K}) > 1$, the verifier uniformly samples and sends a prime q' from a fixed set of primes $\mathcal{P}$ ($\mathcal{P}$ is a protocol parameter), and an element $\alpha \in \mathbb{F}_{q'}$. Now update

$$\begin{aligned} \mathbf{v}^{(1)} &\leftarrow \pi_{q', \alpha} \left(\pi_{\text{canon}}^{-1} \left(\mathbf{v}^{(1)} \right) \right) \in \mathbb{F}_{q'}, \\ \mu_j &\leftarrow \pi_{q'}^{-1}(\pi_{q', \alpha}(\mu_j)) \in [0 .. q' - 1] \subseteq \mathbb{Z}, \quad j \in [\ell_2], \\ q &\leftarrow q'. \end{aligned}$$

2. Phase 2: Transforming the claim into a PESAT claim over $\mathbb{F}_{2^n}$, and invoking the IOR Π_{red} .

- Note that, at this point of the protocol, $\mathbf{v}^{(1)} \in \mathbb{F}_q^{\ell_1}$. Prover and verifier define

$$\gamma = \pi_q^{-1}(\mathbf{v}^{(1)}) \in [0 .. q - 1]^{\ell_1} \subseteq \mathbb{Z}^{\ell_1},$$

and set to prove the following claim:

$$\prod_{i \in [\ell_1]} g^{\gamma_i \cdot \mathbf{f}_{ij}} = g^{\mu_j} \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2].$$

This claim is equivalent to

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2].$$

Notice that

$$(\mathbf{x}_j = ((g^{\gamma_i})_{i \in [\ell_1]}, g^{\mu_j}), \mathbf{y}_j = f_{*j}, \mathbf{w}_j = \pi_2(\mathbf{f}_{*j})) \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}_2\mathbb{Z}}, \mathcal{C}_1, \text{Pack}_n) \quad \text{for all } j \in [\ell_2]. \quad (3)$$

- To prove the above claim, prover and verifier engage in the IOR $\Pi_{\text{red}}^{\ell_2}$ with input

$$(\mathbf{x}_j = ((g^{\gamma_i})_{i \in [\ell_1]}, g^{\mu_j}), \mathbf{y}_j = f_{*j}, \mathbf{w}_j = \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]},$$

where in the honest case $f_{*j} = \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j}))) \in \mathbb{F}_{2^n}^{m/\ell_2}$. At the end of this IOR, they are left with the following claim to prove:

$$((\mathbf{v}, \mu'_j), f_{*j}, \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]} \in (\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^n}, \mathcal{C}_1, \text{Pack}_n))^{\ell_2} \quad (4)$$

for some vector $\mathbf{v}$ and field elements μ'_j output during the IOR $\Pi_{\text{red}}^{\ell_2}$.

3. Phase 2: Using an IOPP over $\mathbb{F}_{2^n}$. Prover and verifier engage in an IOP to certify the claims (??).

²Recall that, in our instantiation, $\Pi_{\text{red}}^{\ell_2}$ is the GKR protocol applied to multiple grand products, cf. ??

Albert: Three fixes flagged while implementing this construction (steps counted as in ??, so the q' -sampling is Step 3): (a) after the Step-3 update, the claim $\prod_{i \in [\ell_1]} g^{\gamma_i \cdot \mathbf{f}_{ij}} = g^{\mu_j}$ is unsatisfiable for an honest prover – the updated μ_j is reduced into $[0..q' - 1]$, while the product's exponent $\langle \mathbf{f}_{*j}, \gamma \rangle$ is only congruent to it mod q' . Restate Step 4 with the prover sending the exact integer fold $\mu'_j = \langle \mathbf{f}_{*j}, \gamma \rangle$ and the verifier checking $\mu'_j \equiv \mu_j(\alpha) \pmod{q'}$ (μ_j = the Step-1 polynomial, α = the Step-3 point); this is what the implementation does. (b) In Step 1 the coefficient range of μ_j should be $[0.. \ell_1(q-1)]$ (a sum of ℓ_1 lifted values), degree $\leq \deg(\mathbb{K}) - 1$. (c) Step 3's case split skips $\deg(\mathbb{K}) = 1$ – it should read “if $\deg(\mathbb{K}) \leq 1$, continue”.

3.4 Soundness and completeness

Theorem 3.2 (Completeness and round-by-round knowledge soundness of ??). *Assume $\Pi_{\text{red}}^{\ell_2}$ is a perfectly complete t_{red} -round IOR from $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}2\mathbb{Z}}, \mathcal{C}_1, \text{Pack}_n)^{\ell_2}$ to $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^n}, \mathcal{C}_1, \text{Pack}_n)^{\ell_2}$ with round-by-round knowledge soundness errors $(\kappa_{\text{red}1}, \dots, \kappa_{\text{red}t_{\text{red}}})$. Assume Π_{iopp} is a perfectly complete t_{iopp} -round IOPP Albert: IOPP or IOP? from $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^n}, \mathcal{C}_1, \text{Pack}_n)^{\ell_2}$ with round-by-round knowledge soundness errors $\kappa_{\text{iopp}1}, \dots, \kappa_{\text{iopp}t_{\text{iopp}}}$.*

Then, if $\deg(\mathbb{K}) > 0$, ?? is a perfectly complete $1 + t_{\text{red}} + t_{\text{iopp}}$ IOPP for $\mathcal{R}_{\text{Lin-BitsDual}}$ with round-by-round knowledge soundness errors $\kappa_0, (\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]}$, where

$$\kappa_0 = \frac{\deg(\mathbb{K})}{q_{\min}} + \frac{\log(\ell_1 + 1) + \log(q - 1)}{|\mathcal{P}| \cdot \log(q_{\min})}$$

and $q_{\min}$ is the smallest prime in the set of sampleable primes $\mathcal{P}$ from Step 3 of ??.

If $\deg(\mathbb{K}) = 0$, i.e. $\mathbb{K}$ is a prime field, then ?? is a $t_{\text{red}} + t_{\text{iopp}}$ IOPP with round-by-round knowledge soundness errors $\kappa_0, (\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]}$.

To prove ?? we begin with the following lemma, which is implicit in [zincplus] Albert: or is it explicit as a particular case of reductions on $\mathbb{Q}[X]$?

Lemma 3.3. *Let $h(X) \in \mathbb{Z}[X]$ be a nonzero polynomial of degree $\leq d$ and with coefficients in the range $[-2^B + 1..2^B - 1]$. Let $\mathcal{P}$ be a set of primes, and let $q_{\min}$ be the smallest prime in $\mathcal{P}$. Then*

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q}} [\pi_{q,\alpha}(f(X)) = 0] \leq \frac{B}{\lfloor \log(q_{\min}) \rfloor \cdot |\mathcal{P}|} + \frac{d}{q_{\min}}.$$

Proof. Let $\mathcal{E}_{\text{prime}}$ be the event that $f(X)$ is a multiple of q . Then, by Schwartz-Zippel lemma,

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q}} [\pi_{q,\alpha}(f(X)) = 0 \mid \neg \mathcal{E}_{\text{prime}}] \leq \frac{d}{q_{\min}}.$$

Now, every nonzero coefficient of $f(X)$ is divisible by at most $B/\lfloor \log(q_{\min}) \rfloor$ primes from $\mathcal{P}$. Hence, the probability that $\mathcal{E}_{\text{prime}}$ occurs is at most $(B/\lfloor \log(q_{\min}) \rfloor)/|\mathcal{P}|$, and the lemma follows. $\square$

Proof of ??. We decompose ?? into three IORs Π_1, Π_2, Π_3 . The first one, Π_1 , is a 1-round IOR from $\mathcal{R}_{\text{Lin-BitsDual}}$ to $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}2\mathbb{Z}}, \mathcal{C}_1, \text{Pack}_n)^{\ell_2}$, and it consists of applying Round 1 of ??, and outputting

$$(\mathbf{x}_j = ((g^{\gamma_i})_{i \in [\ell_1]}, g^{\mu_j}), \mathbf{y}_j = f_{*j}, \mathbf{w}_j = \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]},$$

as described at the beginning of Phase 1 of ???. The second IOR, Π_2 , is simply $\Pi_{\text{red}}^{\ell_2}$, reducing $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}_{2^n}}, \mathcal{C}_1, \text{Pack}_n)^{\ell_2}$ to $(\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n})^{\ell_2}$. Finally, the third IOR, Π_3 , is simply the IOP of proximity Π_{iopp} for $(\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n})^{\ell_2}$.

By the composition of IORs result (cf. ???), it suffices to show that Π_1 has round-by-round knowledge error κ_0 , and is perfectly complete.

Completeness and soundness of Π_1 We describe a knowledge state function KState . Let $\mathbf{x} = (\mathbf{v}, \mu) \in \mathbb{K}^\ell \times \mathbb{K}$ and $\mathbf{y} = f \in \mathbb{F}_{2^n}^m$. Let tr be a partial interaction transcript for ???. We start by setting KState so that the first two conditions in ??? are met. Precisely, if $\text{tr} =$, we set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-BitsDual}}$; and we have KState output zero on inputs $(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \pi, \mathbf{w})$ where π is a prover message [Albert: notation collision](#) and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$.

After the verifier's first message, tr is as follows:

$$\text{tr} = (\mu_1, \dots, \mu_{\ell_2}, q', \alpha) \in [0 \dots q-1]^{\leq \deg(\mathbb{K})} [X]^{\ell_2} \times \mathcal{P} \times \mathbb{F}_{q'}.$$

We define $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$, $\mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in (\{0, 1\}^{\ell_1})^{\ell_2}$, and

$$\begin{aligned} \langle \mathbf{f}_{*j}, \pi_{q'}^{-1}(\pi_{q', \alpha}^{-1}(\pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}))) \rangle &= \pi_{q'}^{-1}(\pi_{q', \alpha}(\mu_j)) \\ f_{*j} &= \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\pi_2((\mathbf{f}_{*j})))) \end{aligned} \quad \text{for all } j \in [\ell_2] \quad (5)$$

[Albert: something about the oracle?](#) We define $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \boldsymbol{\mu} \parallel q', \alpha, \mathbf{w}) = \mathbf{w}$. Let $\boldsymbol{\mu} = (\mu_1, \dots, \mu_{\ell_2})$. We next show that

$$\Pr_{q', \alpha} \left[\exists \mathbf{w} \mid \begin{array}{l} \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \boldsymbol{\mu}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \boldsymbol{\mu}, q', \alpha, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \boldsymbol{\mu}, q', \alpha, \mathbf{w}) = 1 \end{array} \right] \leq \kappa_0(??).$$

Indeed, let $\mathbf{w}' = \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \boldsymbol{\mu}, q', \alpha, \mathbf{w})$. Assume $\text{KState}(\mathbf{x}, \mathbf{y}, \boldsymbol{\mu}, q', \alpha, \mathbf{w}) = 1$ and $\text{KState}(\mathbf{x}, \mathbf{y}, \boldsymbol{\mu}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \boldsymbol{\mu}, q', \alpha, \mathbf{w})) = 0$. The latter occurs if and only if $\text{KState}(\mathbf{x}, \mathbf{y}, \mathbf{w}) = 0$, which in turn occurs if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{Lin-BitsDual}}$. Thus we have to bound the probability over q', α that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{Lin-BitsDual}}$ and $\mathbf{w} = (\mathbf{f}_{*j})$ satisfies ???. To that end, assume $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{Lin-BitsDual}}$. Then $\langle \pi_q(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle \neq \mu_j$ over $\mathbb{K}$ for some $j \in [\ell_2]$. In particular, $\langle \mathbf{f}_{*j}, \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \rangle \neq \pi_{\text{canon}}^{-1}(\mu_j)$ over $\mathbb{Z}[X]$. Let

$$p_j(X) := \langle \mathbf{f}_{*j}, \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \rangle - \pi_{\text{canon}}^{-1}(\mu_j) \in \mathbb{Z}[X].$$

Then p_j is a nonzero integer polynomial of degree $\leq \deg(\mathbb{K})$ and with coefficients of absolute value at most $\ell_1 \cdot (q-1) + (q-1)$ (here we use that all entries in $\mathbf{f}_{*j}$ are bits). Now, if $\mathbf{f}_{*j}$ satisfies ???, then, applying $\pi_{q'}$ on both sides, reorganizing terms, using that $\pi_{q'}(\mathbf{f}_{*j}) = \pi_{q', \alpha}(\mathbf{f}_{*j})$ and the linearity of $\pi_{q', \alpha}$,

$$0 = \langle \pi_{q'}(\mathbf{f}_{*j}), \pi_{q', \alpha}(\pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)})) \rangle - \pi_{q', \alpha}(\mu_j) = \pi_{q', \alpha} \left(\langle \mathbf{f}_{*j}, \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \rangle - \mu_j \right) = \pi_{q', \alpha}(p_j(X)).$$

Hence

$$\Pr_{q', \alpha} [(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{Lin-BitsDual}}, \text{ and } \mathbf{w} = (\mathbf{f}_{*j}) \text{ satisfies } ???] \leq \Pr_{q', \alpha} [\pi_{q', \alpha}(p_j(X)) = 0]$$

By ???,

$$\Pr_{q', \alpha} [\pi_{q', \alpha}(p_j(X)) = 0] \leq \frac{\log(\ell_1 + 1) + \log(q-1)}{\lfloor \log(q_{\min}) \rfloor \cdot |\mathcal{P}|} + \frac{\deg(\mathbb{K})}{q_{\min}} = \kappa_0,$$

as needed.

It remains to see that if $\mathbf{tr}$ is a full transcript of Π_1 , then $\mathbf{KState}(\mathbf{x}, \mathbf{y}, \mathbf{tr}, \mathbf{w}) = 1$ if and only if verifier with input $(\mathbf{x}, \mathbf{y}, \mathbf{tr})$ outputs $(\mathbf{x}', \mathbf{y}')$ such that $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}2\mathbb{Z}}, \mathcal{C}_1, \text{Pack}_n)^{\ell_2}$ [Albert: yes?](#). The output of the verifier is

$$(\mathbf{x}_j = ((g^{\gamma_i})_{i \in [\ell_1]}, g^{\mu_j}), \mathbf{y}_j = f_{*j},)_{j \in [\ell_2]},$$

where

$$\gamma = \pi_{q'}^{-1}(\pi_{q', \alpha}(\pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}))) \in [0 .. q' - 1]^{\ell_1} \subseteq \mathbb{Z}^{\ell_1}.$$

Now, write as before $\mathbf{tr} = (\boldsymbol{\mu}, q', \alpha)$. By definition, $\mathbf{KState}(\mathbf{x}, \mathbf{y}, \mathbf{tr}, \mathbf{w}) = 1$ if and only if $\mathbf{w} = (\mathbf{f}_{*j})_{j \in [\ell_2]}$ satisfies (??). The equalities in (??) take place in $\mathbb{Z}$. Concretely, the right-hand-side is an integer in the range $[0 .. q' - 1]$, and hence so is the left-hand-side. Hence if the multiplicative order of g (the generator of $\mathbb{F}_{2^n}^*$) is larger than $q' - 1$, we have that (??) holds if and only if [Albert: shouldn't we account for the growth \$\ell_1 \cdot \(q' - 1\)\$? Also how does the need of having many samplable primes affect the maximum possible size of \$q'\$?](#)

$$\begin{aligned} g^{\langle \mathbf{f}_{*j}, \gamma \rangle} &= g^{\pi_{q'}^{-1}(\pi_{q', \alpha}(\mu_j))} \\ f_{*j} &= \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\pi_2((\mathbf{f}_{*j})))) \end{aligned} \quad \text{for all } j \in [\ell_2].$$

The above holds if and only if $(\mathbf{x}_j, \mathbf{y}_j, \mathbf{f}_{*j})_{j \in [\ell_2]} \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^n}, Q_{\mathbb{F}2\mathbb{Z}}, \mathcal{C}_1, \text{Pack}_n)$. $\square$

3.5 $\mathbb{F}2\mathbb{Z}$: an IOP for Lin-ModFieldBin

We now describe our IOP for $\mathcal{R}_{\text{Lin-ModFieldBin}}$.

Construction 3.4 (IOP for $\mathcal{R}_{\text{Lin-ModFieldBin}}$). The prover and verifier receive $\mathbf{x} = (\mathbf{v}, \mu) \in \mathbb{K}^{\ell_0} \times \mathbb{K}$ and $\mathbf{y} = f \in \mathbb{F}_{2^n}^m$. The prover receives [Albert: \$\ell_0\$ would be more consistent with the beginning of the section](#) $\mathbf{w} = \mathbf{f} \in M_{R \leq B}^{\ell_0} \subseteq \mathbb{Z}^{\ell_0}$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-ModFieldBin}}$, i.e.,

$$f = \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\text{bits}(\mathbf{f})))), \quad \langle \psi_M(\mathbf{f}), \mathbf{v} \rangle = \mu.$$

1. Rewriting $\langle \psi_M(\mathbf{f}), \mathbf{v} \rangle$, we obtain [Albert: maybe write in a separate lemma, or give more detail?](#)

$$\begin{aligned} \langle \psi_M(\mathbf{f}), \mathbf{v} \rangle &= \langle \psi_M \left(\sum_{\substack{k \in [B], \\ j \in [d]}} \mathbf{f}_{*jk} \cdot 2^{k-1} \cdot \gamma_j \right), \mathbf{v} \rangle = \langle \sum_{\substack{k \in [B], \\ j \in [d]}} \pi_q(\mathbf{f}_{*jk}) \cdot \pi_q(2^{k-1}) \cdot \psi_M(\gamma_j), \mathbf{v} \rangle, \\ &= \sum_{\substack{k \in [B], \\ j \in [d]}} \pi_q(2^{k-1}) \cdot \psi_M(\gamma_j) \cdot \langle \pi_q(\mathbf{f}_{*jk}), \mathbf{v} \rangle \\ &= \sum_{\substack{k \in [B], \\ j \in [d]}} \sum_{i \in [\ell_0]} \pi_q(2^{k-1}) \cdot \psi_M(\gamma_j) \cdot \pi_q(\mathbf{f}_{*jk}), \mathbf{v}_i \\ &= \langle \pi_q(\text{bits}(\mathbf{f})), \mathbf{u} \rangle \\ \mathbf{u} &:= (\mathbf{v} \otimes (1, \pi_q(2), \dots, \pi_q(2^{B-1})) \otimes (\psi_M(\gamma_1), \dots, \psi_M(\gamma_d))) \in \mathbb{K}^{\ell_0 \cdot d \cdot B}. \end{aligned} \tag{6}$$

And so the claim $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{Lin-ModFieldBin}}$ is equivalent to the claim

$$(\mathbf{x}' = (\mathbf{u}, \mu), \mathbf{y}' = f = \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\text{bits}(\mathbf{f})))), \mathbf{w}' = \text{bits}(\mathbf{f})) \in \mathcal{R}_{\text{Lin-BitsDual}}. \quad (7)$$

Accordingly, prover and verifier use the $\mathbb{F}_2\mathbb{Z}$ PCS ?? to prove (??).

Theorem 3.5. *Albert: completeness and soundness*

4 [Old] Extensions, variations, and optimizations

Albert: Section is relatively outdated

4.1 Virtualization of $\mathbb{F}_2$ -linear combinations lifted to integers

Albert: In current implementation, this performs worse than not using any virtualization, which should not be the case. But before working on this section further I'd want to make sure it is worth it

In this section we expand relation $\mathcal{R}_{\text{Lin-BitsDual}}$ to impose, instead of constraints of the form $\text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f$ and $\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu$, constraints as $\text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f$ and

$$\langle \psi(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))), \mathbf{v} \rangle = \mu$$

for a matrix $M \in \mathbb{F}_2^{\ell' \times \ell}$. Precisely, given a new parameter ℓ' , we define

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{K}, \text{Pack}_n}^{\text{lin-virtual}} := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}, M) \in \mathbb{K} \times \mathbb{K}^{\ell'} \times \mathbb{F}_2^{\ell' \times \ell}, \\ \mathbf{y} = f \in \mathbb{F}_{2^n}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^{\ell} \subseteq \mathbb{Z}^{\ell} \end{array} \right) \mid \begin{array}{l} \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f \\ \wedge \langle \psi(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))), \mathbf{v} \rangle = \mu \end{array} \right\}$$

Albert: We could replace $M \cdot \pi_2(\mathbf{f})$ by the output of a GKR circuit on $\pi_2(\mathbf{f})$ The relation has the tag “lin(ear)-virtual” to reference the intuition that $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{K}, \text{Pack}_n}^{\text{lin-virtual}}$ expresses a linear constraint on the vector $\mathbf{h} = \psi(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))) \in \mathbb{K}^{\ell'}$, which is not committed directly: only $\pi_2(\mathbf{f})$ is. Note that $\mathbf{h}$ is a linear expression over $\mathbb{F}_{2^n}$ of the elements in $\pi_2(\mathbf{f})$, lifted to the integers, and then projected to $\mathbb{K}$.

Albert: Would be good to add more detail in this paragraph The relation $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{K}}^{\text{lin-virtual}}$ allows to, for example, express linear constraints over $\mathbb{K}$ of the XOR of witness elements, while only committing (over $\mathbb{F}_{2^n}$) to the original elements. The same applies with other bit operations such as AND and Majority, by noting first that, given integer bits $x, y, z \in \{0, 1\}$: Albert: Should reference some identity in preliminaries, or explain more here

$$\begin{aligned} \psi(x \text{ AND } y) &= 2^{-1} \cdot \psi(x + y - \pi_2^{-1}(\pi_2(x) + \pi_2(y))), \\ \psi(\text{Maj}(x, y, z)) &= 2^{-1} \cdot \psi(x + y + z - \pi_2^{-1}(\pi_2(x) + \pi_2(y) + \pi_2(z))), \end{aligned}$$

where 2^{-1} denotes the inverse of 2 modulo q . Hence AND and Majority can be expressed over $\mathbb{F}_q$ as linear expressions of a vectors of the form $\psi(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f})))$, with $\mathbf{f}$ being a vector of integer bits containing only the original elements on which we apply the AND and Majority operations.

We next provide an IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{K}, \text{Pack}_n}^{\text{lin-virtual}}$ for specific shapes of M . Similarly as in ??, we assume a tensor structure $\ell = \ell_1 \cdot \ell_2$ for $\mathbf{v}$, and write $\mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \{0, 1\}^{\ell_1 \times \ell_2}$. We then let ℓ'_1 be

such that $\ell' = \ell'_1 \times \ell_2$ and assume $M = \text{Id} \otimes M_0$ where M_0 is a $\ell'_1 \times \ell_1$ matrix with entries in $\mathbb{F}_2$, and Id is the $\ell_2 \times \ell_2$ matrix. In words, M is the $\ell' \times \ell = \ell'_1 \cdot \ell_2 \times \ell_1 \cdot \ell_2$ diagonal block matrix with M_0 as diagonal blocks. Then

$$M \cdot \pi_2(\mathbf{f}) = (M_0 \cdot \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]} \in (\mathbb{F}_2^{\ell'_1})^{\ell_2}. \quad (8)$$

Construction 4.1 (IOPP for $\mathcal{R}_{C, \mathbb{F}_{2^n}, \mathbb{K}}^{\text{lin-virtual}}$). The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}, M) \in \mathbb{K} \times \mathbb{K}^{\ell'} \times \mathbb{F}_2^{\ell' \times \ell}$ and $\mathbf{y} = f \in \mathbb{F}_{2^n}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{C, \mathbb{F}_{2^n}, \mathbb{K}}^{\text{lin-virtual}}$. We assume M is of the form described above.

Define $\mathbf{h} = \pi_2^{-1}(M \cdot \pi_2(\mathbf{f})) \in \{0, 1\}^{\ell'} \subseteq \mathbb{Z}^{\ell'}$. By (??),

$$\mathbf{h} = (\mathbf{h}_{*j})_{j \in [\ell_2]} = (\pi_2^{-1}(M_0 \cdot \pi_2(\mathbf{f}_{*j})))_{j \in [\ell_2]},$$

with $\mathbf{h}_{*j} \in \{0, 1\}^{\ell'_1}$. Let $f = \text{Enc}_C(\text{Pack}_n(\pi_2(\mathbf{f}))) \in \{0, 1\}^m \subseteq \mathbb{Z}^m$.

1. Prover and verifier execute ?? with inputs

$$\mathbf{x}' = (\mu, \mathbf{v}), \quad \mathbf{y}' = f, \quad \mathbf{w}' = \mathbf{h} = \pi_2^{-1}(M \cdot \pi_2(\mathbf{f})) \in \{0, 1\}^{\ell'} \subseteq \mathbb{Z}^{\ell'}.$$

until Step 3 where they are meant to use the IOR Π_{red} . Note that until that point, the verifier has not made any queries to $\mathbf{y}'$. So, in terms of completeness, it does not matter, yet, that f is not the encoding of $\pi_2(\mathbf{h}) = M \cdot \pi_2(\mathbf{f})$.

2. Prover and verifier engage in $\Pi_{\text{red}}^{\ell_2}$ with inputs

$$\left(\mathbf{x}_j = ((g^{\gamma_i})_{i \in [\ell'_1]}, g^{\mu_j}), \mathbf{y}_j = f_{*j}, \mathbf{w}_j = \pi_2(\mathbf{h}_{*j}) = M_0 \cdot \pi_2(\mathbf{f}_{*j}) \right)_{j \in [\ell_2]},$$

where $f_{*j} = \text{Enc}_{C_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j})))$. At the end of $\Pi_{\text{red}}^{\ell_2}$ they are left with claims

$$((\mathbf{v}, \eta_j), \mathbf{y}_j, \mathbf{w}_j)_{j \in [\ell_2]} \in (\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^n}, C_1, \text{Pack}_n))^{\ell_2}, \quad (9)$$

for some $\mathbf{v} \in \mathbb{F}_{2^n}^{\ell'_1}$ and $\eta_j \in \mathbb{F}_{2^n}$. Again, since we assume the verifier never queries $\mathbf{y}_j$ during Π_{red} , completeness holds for honest provers, even if $\mathbf{y}_j$ is not the encoding of $\mathbf{w}_j$.

Note that (??) holds if and only if, for all $j \in [\ell_2]$,

$$\begin{aligned} \eta_j &= \sum_{i \in [\ell'_1]} \mathbf{v}_i \cdot \pi_2(\mathbf{h}_{ij}) = \sum_{i \in [\ell'_1]} \mathbf{v}_i \cdot \sum_{t \in [\ell_1]} M_{0it} \cdot \pi_2(\mathbf{f}_{tj}) \\ &= \sum_{t \in [\ell_1]} \left(\sum_{i \in [\ell'_1]} M_{0it} \cdot \mathbf{v}_i \right) \cdot \pi_2(\mathbf{f}_{tj}) = \langle \mathbf{u}_j, \pi_2(\mathbf{f}_{*j}) \rangle, \end{aligned}$$

where $\mathbf{u} \in \mathbb{F}_{2^n}^{\ell'_1}$ is the vector whose t -th entry is

$$\mathbf{u}_t = \langle \mathbf{v}, M_{0*t} \rangle \in \mathbb{F}_{2^n},$$

and $M_{0*t} \in \mathbb{F}_{2^n}^{\ell'_1}$ denotes the t -th column of M_0 , for $t \in [\ell_1]$,

Note that, if $\Pi_{\text{red}}^{\ell_2}$ is as in ??, then $\mathbf{v} = (\text{eq}(\mathbf{b}, \mathbf{r}))_{\mathbf{b} \in \{0, 1\}^{\log \ell'_1}}$ for some $\mathbf{r} \in \mathbb{F}_{2^n}^{\log \ell'_1}$, and then $\mathbf{u}_t = \text{mle}[M_{0*t}](\mathbf{r}) \in \mathbb{F}_{2^n}$.

Theorem 4.3. *Albert: Completeness and RBR knowledge soundness of ??*

Proof. □

4.1.1 Optimizing $\mathbb{F}2\mathbb{Z}$ for virtualized inputs $\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))$

In ??, at Step 2, prover and verifier use $\Pi_{\text{red}}^{\ell_2}$ to reduce the claim

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot (M_0 \cdot \pi_2(\mathbf{f}_{*j}))_i + 1) = g^{\mu_j} \quad \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2] \quad (12)$$

to ℓ_2 inner product claims over $\mathbb{F}_{2^n}$ for the vectors $M_0 \cdot \pi_2(\mathbf{f}_{*j})$. In many applications of interest (see ??), the vectors $M_0 \cdot \pi_2(\mathbf{f}_{*j})$ are larger than $\mathbf{f}_{*j}$, but contain entries that are structured expressions of the entries in $\mathbf{f}_{*j}$. For example, in ??, $M \cdot \pi_2(\mathbf{f})$ is the concatenation of $\pi_2(\mathbf{f})$ and the vector with entries $\pi_2(\mathbf{f})_i \text{ XOR } \pi_2(\mathbf{f})_{i+1}$.

A natural question is whether, in such scenarios, the proof of (??) can be optimized so that it costs less than a full proof for ℓ_2 arbitrary input vectors of the same length as $M_0 \cdot \pi_2(\mathbf{f}_{*j})$. In this section we show that this is indeed possible.

4.2 An optimized GKR protocol for the $\mathbb{F}2\mathbb{Z}$ grand products

The bulk of the work in $\mathbb{F}2\mathbb{Z}$ (??) is to reduce the claim

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \quad \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2]. \quad (13)$$

to ℓ_2 inner product claims for the vectors $\pi_2(\mathbf{f}_{*j})$. We do this reduction via the GKR protocol for multiple grand products, and make several implementation-level optimizations for it.

Albert: Todo

Albert: Present the optimization as a contribution of the paper *Albert*: There seems to be a comment in the Binius repo about this though: "TODO: Use a special 1-bit-optimized MLE-check with switchover to save memory" (prove.rs:380), "TODO: Optimize with bit-masking for selection" (witness.rs:188). They also use some similar optimizations but for non-grand product sumchecks. Worth seeing carefully what we are doing vs them

4.3 Proof size optimization for large inputs

When $\ell = \ell_1 \cdot \ell_2$ is such that ℓ_2 is large, say $\ell \geq 2^{11}$, the elements $\mu_1, \dots, \mu_{\ell_2} \in \mathbb{Z}$ sent at Step 1 of ?? may start to dominate the proof size. For example, for $\ell = 2^{13}$ and μ_i of 128 bits, the elements $\mu_1, \dots, \mu_{\ell_2}$ amount to 131 KB. To mitigate this cost, the idea is to have the prover commit to the vector $(\mu_1, \dots, \mu_{\ell_2})$ and then prove (??), i.e.

$$\sum_{j \in [\ell_1]} \psi(\mu_j) \cdot \mathbf{v}_j^{(2)} = \mu \quad \text{over } \mathbb{K}.$$

using an IOPP for constrained codes. Concretely, the prover commits to the vector $\text{bits}(\mu)$ of bits forming the elements $(\mu_1, \dots, \mu_{\ell_2})$ using $\mathbb{F}2\mathbb{Z}$, and then proves (??) using ??. At the end of ??, the prover is left with an inner product claim for the vector $\pi_2(\text{bits}(\mu))$. *Albert: TODO: explain how to batch this claim with the WHIR IOPP for the final step of ??*

5 $\mathbb{F}2\mathbb{Z}$ for PIMSAT

Albert: I'm aiming to make the paper self-contained, so I will refer to Zinc/Zinc+ but will provide all required definitions and proofs in this paper

Albert: In the F2Z world, when elements in the computation never exceed 100 bits, we can work on a fixed prime, instead of a random one, because F2Z enforces range checks automatically

Throughout this section R denotes either $\mathbb{Z}$ or a finite field $\mathbb{F}$ with q^e elements.

5.1 Polynomial Ideal Membership Satisfiability (PIMSAT)

In this section we describe a variation of the PESAT relation (cf. ??), called *Polynomial Ideal Membership Satisfiability (PIMSAT)*. It generalizes PESAT by allowing polynomial constraints over polynomial rings (or modules), and by replacing equality to zero by ideal membership.

We fix a witness length ℓ Albert: for consistency, this would be ℓ_0 in previous sections?, an input length k , a bit size bound B , a degree bound d , a collection of ι distinct ideals $\mathcal{I}_1, \dots, \mathcal{I}_\iota$ of $R[X]$, a number of rows 2^ν , and for each $i \in [\iota]$, a vector of 2^ν polynomials Albert: No need to have number of rows be a power of two

$$\mathbf{p}^{(i)} = \left(\mathbf{p}_{\mathbf{b}}^{(i)} \mid \mathbf{b} \in \{0, 1\}^\nu \right) \in (R_{\leq B}[Y_1, \dots, Y_{k+\ell}])^{2^\nu}$$

with coefficients in $R_{\leq B}$ and on $k + \ell$ variables. The PIMSAT relation with respect to these parameters is defined as

$$\mathcal{R}_{\text{PIMSAT}} = \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \left(R_{\leq B}^{\leq d}[X] \right)^k, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in \left(R_{\leq B}^{\leq d}[X] \right)^\ell \end{array} \mid \mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f}) \in \mathcal{I}_i \text{ for all } \mathbf{b} \in \{0, 1\}^\nu, i \in [\iota] \right\}$$

The next remark explains that all our constructions for $\mathcal{R}_{\text{PIMSAT}}$ apply, in particular, to the usual R1CS, CCS [ccs], AIR [air], lookups, permutation constraints, etc. over finite fields.

Albert: Motivate ideals somewhere (intro?)

Example 5.1 (R1CS constraints over $R[X]$ with ideal membership). Let $R, k, \ell, 2^\nu, \iota, \mathcal{I}_1, \dots, \mathcal{I}_\iota, B, d$ be as before, and let $A^{(i)}, B^{(i)}, C^{(i)}$ be $2^\nu \times (k + \ell)$ matrices with entries in $R_{\leq B}^{\leq d}[X]$ (typically, these are sparse matrices with entries in a small subset of R), for all $i \in [\iota]$. Then the R1CS-PIMSAT relation is defined as:

$$\mathcal{R}_{\text{R1CS-PIMSAT}} := \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \left(R_{\leq B}^{\leq d}[X] \right)^k, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in \left(R_{\leq B}^{\leq d}[X] \right)^\ell \end{array} \mid \begin{array}{l} \left(\left(A^{(i)} \cdot \mathbf{z} \right) \circ \left(B^{(i)} \cdot \mathbf{z} \right) - \left(C^{(i)} \cdot \mathbf{z} \right) \right)_{\mathbf{b}} \in \mathcal{I}_i \\ \text{for all } \mathbf{b} \in \{0, 1\}^\nu, i \in [\iota] \end{array} \right\},$$

where $\text{int}(\mathbf{b})$ denotes the nonnegative integer whose bit-representation is $\mathbf{b}$. The relation $\mathcal{R}_{\text{R1CS-PIMSAT}}$ is the subset of $\mathcal{R}_{\text{PIMSAT}}$ where

$$\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f}) = \left(\left(A^{(i)} \cdot \mathbf{z} \right) \circ \left(B^{(i)} \cdot \mathbf{z} \right) - \left(C^{(i)} \cdot \mathbf{z} \right) \right)_{\text{int}(\mathbf{b})}.$$

Following ?? and [linear-accumulation; zinc; zincplus] one can show that PIMSAT allows to express popular constraint systems such as CCS, AIR, and lookup and permutation constraints, over polynomial rings and with ideal membership.

Remark 5.2 (PIMSAT vs the UCS relation from Zinc+ [zincplus]). PIMSAT is a particular subset of the widely general UCS relation from [zincplus]. In this work, we will build a IOP for PIMSAT with witnesses being committed over binary fields. The same ideas can be used to construct an analogous IOP for UCS with witnesses with entries in a bounded subset of $\mathbb{Z}[X]$ (as opposed to $\mathbb{Q}[X]$ Albert: committing to rational through their bit-decomposition is a bit strange, but it may still be possible?). We choose to focus on PIMSAT to favor simplicity of exposition and of our IOP.

The main difference between PIMSAT and UCS is that the later allows for witnesses and constraints over arbitrary and multiple domains in the same instance, including rational polynomial rings $\mathbb{Q}[X]$, and it allows to reuse witness elements in $\mathbb{Q}[X]$ or $\mathbb{Z}[X]$ in other domains such as finite fields and polynomial rings over finite fields, by using homomorphic projections.

5.2 $\mathbb{F}_2\mathbb{Z}$ -IOP: an IOP for PIMSAT with oracles in binary fields

We now describe an IOP for $\mathcal{R}_{\text{PIMSAT}}$. We let $\mathcal{C}$ be a linear code over $\mathbb{F}_{2^n}$ of dimension $(\ell \cdot (d+1) \cdot B)/n$ and blocklength m . We also let, for each finite field $\mathbb{K}$, $\Pi_{\mathbb{K}}$ be an IOR that given $(\mathbf{x}, \perp, \mathbf{w})$ for the PESAT relation (cf. ??) over $\mathbb{K}$, outputs a MLE evaluation claim for $\mathbf{w}$. In other words, it outputs $((\mathbf{r}, \mu), \perp, \mathbf{w})$ in

$$\mathcal{R}_{\text{mle}} = \left\{ ((\mathbf{r}, \mu), \perp, \mathbf{f}) \in \mathbb{K}^{\log \ell + 1 + \ell} \mid \text{mle}[\mathbf{f}](\mathbf{r}) = \mu \right\}$$

Construction 5.3. [IOP for $\mathcal{R}_{\text{PIMSAT}}$] The prover and verifier receive $\mathbf{x} = \mathbf{x} \in R_{\leq B}^{\leq d}[X]^k$. The prover additionally receives $\mathbf{w} = \mathbf{f} \in R_{\leq B}^{\leq d}[X]^\ell$ such that $(\mathbf{x}, \perp, \mathbf{w}) \in \mathcal{R}_{\text{PIMSAT}}$.

1. The prover computes

$$f = \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\text{bits}(\mathbf{f})))) \in \mathbb{F}_{2^n}^m$$

(cf. ??) and sends to the verifier oracle access to f .

2. The prover forms the multilinear polynomial, for all $i \in [\iota]$,

$$Q^{(i)}(Z_1, \dots, Z_\nu) := \sum_{\mathbf{b} \in \{0,1\}^\nu} \mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{z}, \mathbf{f}) \cdot \text{eq}(\mathbf{b}, Z_1, \dots, Z_\nu) \in (R[X])[Z_1, \dots, Z_\nu].$$

The claim that $\mathbf{p}_{\mathbf{b}}^{(i)}(\mathbf{x}, \mathbf{f}) \in \mathcal{I}_i$ for all $\mathbf{b} \in \{0,1\}^\nu$ and $i \in [\iota]$ is equivalent to the claim

$$Q^{(i)}(\mathbf{z}) \in \mathcal{I}_i \quad \text{for all } \mathbf{z} \in \{0,1\}^\nu, i \in [\iota]. \quad (14)$$

3. If $R = \mathbb{F}$, we let $\mathbb{K}$ be a large enough (to be determined later) extension of $\mathbb{F}$. If $R = \mathbb{Z}$, verifier samples and sends a large random field which we denote, abusing the notation, $\mathbb{K}$. Let $q^{e'}$ be the size of $\mathbb{K}$ in either case, for q prime.

The claim (??) is replaced by

$$\pi_q(Q^{(i)}(\mathbf{z})) \in \pi_q(\mathcal{I}_i) \quad \text{for all } \mathbf{z} \in \{0,1\}^\nu, i \in [\iota]. \quad (15)$$

Recall (cf. ??) that π_q is coefficient-wise reduction modulo q when $R = \mathbb{Z}$, and it is the identity map when R is a field $\mathbb{F}$ of characteristic q .

4. The verifier samples and sends a random point $\mathbf{r} \in \mathbb{K}^\nu$.
5. The prover responds with values $u_i \in \mathbb{K}[X]$ for each $i \in [\ell]$. Supposedly, $u_i = \pi_q(Q^{(i)})(\mathbf{r})$. *Albert: Explain somewhere that if $\mathbf{p}'$ is linear, then u_i is easy to compute*
6. The verifier checks that $u_i \in \pi_q(\mathcal{I}_i)$ *Albert: discuss its definition?* for all $i \in [\ell]$. Then, if $d > 0$, it samples and sends elements $\alpha, \theta_1, \dots, \theta_\ell \in \mathbb{K}$. *Albert: Optimization: when there's only one ideal and this is $(X - c)$, we can fix $\alpha = c$ (though in that case the initial constraint can easily be written without ideals at all)*
7. Let $\pi := \pi_{q,\alpha}$ (cf. ??). The prover and verifier replace the claim (??) with the claim

$$\sum_{i \in [\ell]} \sum_{\mathbf{b} \in \{0,1\}^\nu} \theta_i \cdot \pi(\mathbf{p}_{\mathbf{b}}^{(i)})(\pi(\mathbf{x}), \pi(\mathbf{f})) \cdot \text{eq}(\mathbf{b}, \mathbf{r}) = \sum_{i \in [\ell]} \theta_i \cdot \pi(u_i) \in \mathbb{K}. \quad (16)$$

The claim (??) forms a valid instance-witness for the PESAT relation (cf. ??) $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, \mathbf{p})$ with one polynomial constraint $\mathbf{p} = (\mathbf{p}_1)$ given by

$$\mathbf{p}_1 = \sum_{i \in [\ell]} \theta_i \cdot \sum_{\mathbf{b} \in \{0,1\}^\nu} \pi(\mathbf{p}_{\mathbf{b}}^{(i)})(Y_1, \dots, Y_{k+\ell}) - \sum_{i \in [\ell]} \gamma_i \cdot u'_i,$$

with public input $\pi(\mathbf{x})$, and witness $\pi(\mathbf{f})$. Accordingly, prover and verifier use the IOR $\Pi_{\mathbb{K}}$ to reduce (??) to a claim of the form

$$\text{mle}[\pi(\mathbf{f})](\mathbf{r}^{(2)}) = \mu \quad (17)$$

for some $\mathbf{r}^{(2)} \in \mathbb{K}^{\log \ell}$ *Albert: Ask ℓ to be a power of two before* and some $\mu \in \mathbb{K}$. In other words, $((\mathbf{r}^{(2)}, \mu), \perp, \pi(\mathbf{f})) \in \mathcal{R}_{\text{mle}}$ over $\mathbb{K}$.

8. The initial claim $(\mathbf{x}, \perp, \mathbf{w}) \in \mathcal{R}_{\text{PIMSAT}}$ has been replaced by $\text{mle}[\pi(\mathbf{f})](\mathbf{r}^{(2)}) = \mu$. Additionally, verifier now has oracle access to $f = \text{Enc}_C(\text{Pack}_n(\pi_2(\text{bits}(\mathbf{f})))$. Notice that $\pi : R[X] \rightarrow \mathbb{K}$ induces a π_q -module homomorphism $\psi : R^{\leq d}[X] \rightarrow \mathbb{K}$ given by $\psi(h(X)) = \pi(h(X))$. Hence prover and verifier can now use ?? to prove that

$$((\mathbf{v}, \mu), f, \mathbf{f}) \in \mathcal{R}_{\text{Lin-ModFieldBin}}$$

where $\mathbf{v} = (\text{eq}(\mathbf{b}, \mathbf{r}^{(2)}))_{\mathbf{b} \in \{0,1\}^{\log \ell}}$.

Theorem 5.4. *Albert: RBR knowledge soundness and completeness*

Proof. *Albert: Sketch* The plan: the above IOP is the result of compiling a PIOP Π_{PIOP} (given by Steps 2 to 8) with an IOP $\Pi_{\mathbb{F}2\mathbb{Z}}$ for committing to $\mathbf{f} \in R_{\leq B}^{\leq d}[X]$ and proving $\text{mle}[\pi(\mathbf{f})](\mathbf{r}^{(2)}) = \mu$. The PIOP Π_{PIOP} is a particular case of the Zinc+ PIOP, so RBR soundness and completeness can be obtained from there *Albert: Would be preferable to have a self-contained paper (maybe?) so write the proof in the appendix, briefly.* The IOP $\Pi_{\mathbb{F}2\mathbb{Z}}$ has already been studied, except that we started with $\mathbf{f}$ containing only bits, so we need to add a wrapper around it *Albert: todo.*

The compilation is done and its soundness proved in Zinc+, so we can also cite that. There's the small issue that the compiler in Zinc+ assumes the IOPP is using an interleaved code over R to "commit" to the witness, so we cannot 100% black-box the result from Zinc+, though a small remark may do it *Albert: Again, it may be good to have the proof here in an appendix.* $\square$

Remark 5.5. Albert: Scratch? We discuss the specific case where

$$\mathbf{p}_{\mathbf{b}}^{(i)}(Y_1, \dots, Y_{k+\ell}) = \mathbf{p}'^{(i)}(\mathbf{b}, Y_1, \dots, Y_{k+\ell}) \quad \text{for all } i \in [\iota],$$

i.e. there is a $\nu + k + \ell$ -variate polynomial $\mathbf{p}'^{(i)}$ such that the polynomials $\mathbf{p}_{\mathbf{b}}^{(i)}(Y_1, \dots, Y_{k+\ell})$ are obtained by evaluating the first ν variables of $\mathbf{p}'^{(i)}$ on $\mathbf{b} \in \{0, 1\}^\nu$. This is the case in constraints such as R1CS, CCS, and AIR.

....

... The prover and verifier engage in the batched sumcheck protocol over $\mathbb{K}$ for the claim

$$\sum_{\mathbf{b} \in \{0, 1\}^\nu} \pi(\mathbf{p}'^{(i)})(\mathbf{b}, \pi(\mathbf{x}), \pi(\mathbf{f})) \cdot \text{eq}(\mathbf{b}, \mathbf{r}) = \pi(u_i) \in \mathbb{K} \quad \text{for all } i \in [\iota].$$

At the end of batched sumcheck, they are left with a claim of the form

$$\pi(\mathbf{p}'^{(i)})(\mathbf{r}^{(1)}, \pi(\mathbf{x}), \pi(\mathbf{f})) = u'_i \in \mathbb{K} \quad \text{for all } i \in [\iota] \quad (18)$$

for some $\mathbf{r}^{(1)} \in \mathbb{K}^\nu, u'_i \in \mathbb{K}$.

....

...

The claim (??) forms a valid instance-witness for the PESAT relation (cf. ??) over $\mathbb{K}$ with polynomial constraints given by $\pi(\mathbf{p}'^{(i)})(\mathbf{r}^{(1)}, Y_1, \dots, Y_{k+\ell}) - u'_i$, i.e. for such constraints and corresponding parameters

$$(\pi(\mathbf{x}), \perp, \pi(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}.$$

Accordingly, prover and verifier use the IOR $\Pi_{\mathbb{K}}$ to reduce (??) to a claim of the form

$$\text{mle}[\pi(\mathbf{f})](\mathbf{r}^{(2)}) = \mu \quad (19)$$

for some $\mathbf{r}^{(2)} \in \mathbb{K}^{\log \ell}$ **Albert: Ask ℓ to be a power of two before** and some $\mu \in \mathbb{K}$. In other words, $((\mathbf{r}^{(2)}, \mu), \perp, \pi(\mathbf{f})) \in \mathcal{R}_{\text{mle}}$ over $\mathbb{K}$.

5.3 Univariate skip as ideal membership

5.4 Repeated computations through ideal membership

6 Adding Zero Knowledge

6.1 PIOP

6.2 F2Z-PCS

Albert: Sketch. Zk for F2Z is all known territory except Step 1 of ?? where P sends integer (polynomials) $\mu_1, \dots, \mu_{\ell_2}$. Even though $\pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)})$ is uniformly random in $[0, q - 1]$, the elements μ_i do not follow a uniform distribution over any interval in $\mathbb{Z}$.

One way to address this is to have P commit to $\mu_1, \dots, \mu_{\ell_2}$ over $\mathbb{F}_{2^n}$ in zk (it commits to the bits), and to an element $u \in \mathbb{Z}$ such that $\langle \mathbf{f}_{*j}, \mathbb{F}_{2^n}^{-1}(\mathbf{v}^{(1)}) \rangle = \mu_j$ and $\sum_{j \in [\ell_2]} \mu_j \cdot \mathbf{v}_j^{(2)} = q \cdot u$ in $\mathbb{Z}$ (**)

Albert: note that in general we could be working on $\mathbb{Z}[X]$ here, so here we would add $+q \cdot u + h(X) \cdot w$

where $h(X)$ is the irreducible polynomial defining $\mathbb{K}$. Now P proves (**) arithmetized over $\mathbb{F}_{2^n}$, using say Binius64 with zk.

Assume $\deg(\mathbb{K}) = 0$ for now. Then P and V continue with ??, using zk-GKR and zk-Ligerito. At some point P needs to prove that some binary field elements equal $g^{\mu_1}, \dots, g^{\mu_{\ell_2}}$. It does this with one zk-GKR more grand product (this time much smaller).

If $\deg(\mathbb{K}) > 0$, we proceed in the same way, except that the elements $g^{\mu_1}, \dots, g^{\mu_{\ell_2}}$ are instead $g^{\pi_q^{-1}(\pi_{q',\alpha}(\mu_j))}$, and so P must prove that some elements equal $g^{\mu'_j}$, and that $\mu_j = \pi_q^{-1}(\pi_{q',\alpha}(\mu_j))$ in $\mathbb{Z}$. The latter equality consists in evaluating the polynomial μ_j (whose bits we committed to over a binary field) on $X = \alpha'$, reducing mod q' , and lifting to integers. Thus we commit to $\mu'_j \in [0, q' - 1]$ and w'_j and prove that $\mu'_j = \mu_j(\alpha) + q' \cdot w'_j$ (***).

When $\deg(\mathbb{K}) > 0$, it is important to prove (**) and (***) together, since, at least for Binius64, integer multiplication proofs have a floor of about 40-50ms, but scale well. Typical ℓ_2 are 2^8 to 2^{11} . Claims (**) and (***) amount to $2 \cdot \ell_2 * \deg(\mathbb{K}) + 2$ 128-bit multiplications. For $\deg(\mathbb{K}) = 2$ (occurs if we arithmetize our constraints over Goldilocks) and $\ell_2 = 2^{10}$ this is $\sim 2^{12}$ 128-bit integer mults.

Table 2: End-to-end Binius64 proving cost for N independent $128 \times 128 \rightarrow 256$ multiplications, without and with zero knowledge. Each multiplication is composed schoolbook-style from four native $64 \times 64 \rightarrow 128$ imul gates (4 MUL and 11 AND constraints). The ZK path is *recursive*: it symbolically executes the inner verifier to derive an outer Spartan constraint system, pads it with blinding, and commits through the BaseFold ZK compiler. Its overhead is therefore close to *additive* rather than multiplicative — roughly +430 KiB of proof, +2.4 ms of verification and +20–30 ms of proving, independent of N — so the relative cost falls as N grows. Measured on arm64 macOS (NEON+AES, all cores), LOG_INV_RATE = 1, Criterion mid-estimates over 10 samples.

N	Prove (ms)		Verify (ms)		Proof (KiB)		Peak mem (MB)	
	plain	ZK	plain	ZK	plain	ZK	plain	ZK
2^8	41.74	60.09	1.55	3.75	152.22	546.27	38.89	46.04
2^9	46.62	66.03	1.80	4.13	161.58	577.38	47.25	43.55
2^{10}	53.43	74.71	2.12	4.41	172.44	614.23	55.97	51.58
2^{11}	65.69	85.07	2.56	4.91	186.30	643.12	74.20	79.91
2^{12}	78.79	102.35	3.22	5.61	256.94	681.48	148.39	159.35
2^{13}	103.26	134.27	4.29	6.99	273.55	718.34	296.76	318.22

7 Open questions for us

Albert: Regarding ??, I’ve only found a way to cheapen the following settings: say first and two vectors in the grand product are $\mathbf{g}_1, \mathbf{g}_2$, and third is $\mathbf{g}_1 \text{ XOR } \mathbf{g}_2$. This is a tricky problem. We should look more into it

Albert: Should we allow for $W > 1$? Currently just setting $W = 1$ and thinking of all bits as unrelated leads to better performance, but that sounds suboptimal.

8 Case-study

Albert: World passport? Lean circuits? ML models? Hashes?

8.1 Integer and modular multiplication

Albert: We could do something synthetic, like a chain of mults and additions, or a ML or Lean motivated computation

8.2 SHA-256 hashing

SHA-256 compression overview We begin with a brief overview of SHA-256 [FIPS180-4] to establish notation, then formalize the relation we seek to arithmetize.

For the purposes of SHA-256, a *word* is a ≤ 32 string of bits, which we represent as a binary integer polynomial, i.e. as an element of $\{0, 1\}^{<32}[X]$. The SHA-256 compression function receives as input a 512-bit string, represented as sixteen *message words* $(M_t)_{t \in [16]} \in \{0, 1\}^{<32}[X]^{16}$. The algorithm also uses 64 fixed *round constants* $(K_t)_{t \in [64]} \in \{0, 1\}^{<32}[X]^{64}$.

We let

$$\begin{aligned} \psi_2 : \mathbb{Z}[X] &\rightarrow \mathbb{Z} \\ a(X) &\mapsto a(2) \end{aligned} \tag{20}$$

be the ring homomorphism sending integer polynomials to their evaluation at $X = 2$. Notice that, given $a \in \{0, 1\}^{<32}[X]$, the integer $\psi_2(a) \in \mathbb{Z}$ is the integer whose bit-representation coincides with the coefficients of a . The restriction

$$\psi_2 : \{0, 1\}^{<32}[X] \longrightarrow \{0, \dots, 2^{32} - 1\}$$

is a bijection; throughout this subsection, ψ_2^{-1} denotes the inverse of this restricted map, not an inverse on all of $\mathbb{Z}[X]$.

SHA-256 is built from the following six auxiliary functions. For $x, y, z \in \{0, 1\}^{<32}[X]$, define:

$$\begin{aligned} \Sigma_0(x) &:= \text{ROTR}^2(x) \text{ XOR } \text{ROTR}^{13}(x) \text{ XOR } \text{ROTR}^{22}(x) && \in \{0, 1\}^{<32}[X] \\ \Sigma_1(x) &:= \text{ROTR}^6(x) \text{ XOR } \text{ROTR}^{11}(x) \text{ XOR } \text{ROTR}^{25}(x) && \in \{0, 1\}^{<32}[X] \\ \text{Ch}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (\neg x \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\ \text{Maj}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (x \text{ AND } z) \text{ XOR } (y \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\ \sigma_0(x) &:= \text{ROTR}^7(x) \text{ XOR } \text{ROTR}^{18}(x) \text{ XOR } \text{SHR}^3(x) && \in \{0, 1\}^{<32}[X] \\ \sigma_1(x) &:= \text{ROTR}^{17}(x) \text{ XOR } \text{ROTR}^{19}(x) \text{ XOR } \text{SHR}^{10}(x) && \in \{0, 1\}^{<32}[X] \end{aligned} \tag{21}$$

Here ROTR^r rotates a word right by r positions, SHR^r shifts it right by r positions and fills the high positions with zero, and XOR , AND , $\neg$ act coefficient-wise on bits.

Message schedule. At the beginning of the SHA-256 compression function, the sixteen input message words $(M_t)_{t \in [16]}$ are expanded into a so-called *message schedule*, which consists of 64 words $(W_t)_{t \in [64]} \in (\{0, 1\}^{<32}[X])^{64}$ defined as follows:

$$W_t := \begin{cases} M_t & t \in [16], \\ \psi_2^{-1}(\psi_2(W_{t-16}) + \psi_2(\sigma_0(W_{t-15})) + \psi_2(W_{t-7}) + \psi_2(\sigma_1(W_{t-2})) \bmod 2^{32}) & t \in \{17, \dots, 64\}. \end{cases} \tag{22}$$

Compression loop. After the message schedule is computed, SHA-256 performs the so-called *compression loop*. Throughout it, the function maintains an 8-word internal state, conventionally labeled as

$$(a, b, c, d, e, f, g, h) \in \{0, 1\}^{<32}[X]^8.$$

The compression loop iterates 64 rounds. At each round $t \in [64]$, the state entering that round is denoted $(a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t) \in \{0, 1\}^{<32}[X]^8$. The per-round update is given by, for $t \in [64]$,

$$\begin{aligned} T_{1,t} &:= \psi_2^{-1}(\psi_2(h_t) + \psi_2(\Sigma_1(e_t)) + \psi_2(\text{Ch}(e_t, f_t, g_t)) + \psi_2(K_t) + \psi_2(W_t) \bmod 2^{32}), \\ T_{2,t} &:= \psi_2^{-1}(\psi_2(\Sigma_0(a_t)) + \psi_2(\text{Maj}(a_t, b_t, c_t)) \bmod 2^{32}), \\ a_{t+1} &:= \psi_2^{-1}(\psi_2(T_{1,t}) + \psi_2(T_{2,t}) \bmod 2^{32}), \\ e_{t+1} &:= \psi_2^{-1}(\psi_2(d_t) + \psi_2(T_{1,t}) \bmod 2^{32}), \\ (b_{t+1}, c_{t+1}, d_{t+1}, f_{t+1}, g_{t+1}, h_{t+1}) &:= (a_t, b_t, c_t, e_t, f_t, g_t). \end{aligned}$$

For the feed-forward step, let

$$\begin{aligned} S^{\text{in}} &= (a_1, b_1, c_1, d_1, e_1, f_1, g_1, h_1), \\ S^{\text{fin}} &= (a_{65}, b_{65}, c_{65}, d_{65}, e_{65}, f_{65}, g_{65}, h_{65}). \end{aligned}$$

The compression output is the word vector H^{out} defined by

$$H_j^{\text{out}} = \psi_2^{-1}(\psi_2(S_j^{\text{in}}) + \psi_2(S_j^{\text{fin}}) \bmod 2^{32}) \quad (j \in \{0, \dots, 7\}).$$

8.3 Experiments

Albert: GHASH or b127?

Optimization ideas

- b127 instead of GHASH
- Lookup table (algorithm-wise, not SNARK-wise) for first levels of GKR
- If we have an MLE claim of a vector and its shift by one position, can we reuse something for the latter claim from the computations made when proving the former?

8.3.1 Comparison with Hyrax

Albert: Comparing with Hyrax may be more interesting than it sounds initially. Hyrax is pay-per-bit and opens on large prime fields: a profile that F2Z can adopt. Hence (maybe) projects using Hyrax could use F2Z as a drop-in replacement which is PQ secure and more efficient

8.3.2 Comparison with Flock

John: It would be interesting to compare with flock as well

A Ring switching via Galois orbits

Albert: Rewriting the previous appendix for arbitrary inner products

Construction A.1. [Galois ring-switch: an IOR from $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^n}, \mathcal{C}, \text{Pack}_n)$ to $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{id}}^{\text{mle}}$] Albert:
 TODO: formulate as a reduction from $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^n}, \mathcal{C}, \text{Pack}_n)$ to $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{id}}^{\text{mle}}$.

Let $b_1, \dots, b_m \in \mathbb{F}_2^{2^n}$. Using an $\mathbb{F}_2$ -basis $\beta_1, \dots, \beta_m$ of $\mathbb{F}_{2^m}$ we pack m inner products

$$\langle b_i, \mathbf{v} \rangle = \sum_{j \in \{0,1\}^n} b_{i,j} \cdot \mathbf{v}_j, \quad i = 1, \dots, m,$$

coefficient-wise into a single inner product

$$\begin{aligned} b &:= b_1 \cdot \beta_1 + \dots b_m \cdot \beta_m \in \mathbb{F}_{2^m}^{2^n} \\ \langle b, \mathbf{v} \rangle &= \langle b_1, \mathbf{v} \rangle \cdot \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \beta_m \\ &= \sum_{j \in \{0,1\}^n} (b_{1,j} \cdot \beta_1 + \dots + b_{m,j} \cdot \beta_m) \cdot \mathbf{v}_j. \end{aligned}$$

The Frobenius automorphism $\phi(z) = z^2$ keeps $\mathbb{F}_2 \subset \mathbb{F}_{2^m}$ fixed, and has no other fixed points. We extract the value of $\langle b_i, \mathbf{v} \rangle$ from the values of $\langle b, \mathbf{v} \rangle$ along the Galois orbit

$$\mathbf{v}, \mathbf{v}^2, \mathbf{v}^{2^2}, \dots, \mathbf{v}^{2^{m-1}},$$

as follows, where by $\mathbf{v}^t$ we mean that each component of $\mathbf{v}$ is raised to the power t . The Moore matrix

$$M = \begin{pmatrix} \beta_1 & \beta_2 & \dots & \beta_m \\ \beta_1^2 & \beta_2^2 & \dots & \beta_m^2 \\ \beta_1^{2^2} & \beta_2^{2^2} & \dots & \beta_m^{2^2} \\ \vdots & & & \\ \beta_1^{2^{m-1}} & \beta_2^{2^{m-1}} & \dots & \beta_m^{2^{m-1}} \end{pmatrix}$$

consists of the Galois orbits of the basis vectors, and is invertible. (See below.) Using that the component polynomials are over $\mathbb{F}_2$, we see that

$$\phi^k \left(\langle b, \phi^{-k}(\mathbf{v}) \rangle \right) = \langle b_1, \mathbf{v} \rangle \cdot \phi^k \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \phi^k \beta_m,$$

for any k . To this end, observe that

$$M \cdot \begin{pmatrix} \langle b_1, \mathbf{v} \rangle \\ \langle b_2, \mathbf{v} \rangle \\ \langle b_3, \mathbf{v} \rangle \\ \vdots \\ \langle b_m, \mathbf{v} \rangle \end{pmatrix} = \begin{pmatrix} \langle b, \mathbf{v} \rangle \\ \phi \left(\langle b, \phi^{-1}(\mathbf{v}) \rangle \right) \\ \phi^2 \left(\langle b, \phi^{-2}(\mathbf{v}) \rangle \right) \\ \vdots \\ \phi^{m-1} \left(\langle b, \phi^{-(m-1)}(\mathbf{v}) \rangle \right) \end{pmatrix}.$$

Multiplying both sides by M^{-1} we obtain an expression of the values $\langle b_i, \mathbf{v} \rangle$ as a linear expression of the values $\phi^k(\langle b, \phi^{-k}(\mathbf{v}) \rangle)$. **Ulrich:** Invertibility of M follows from that of M^t : any linear functional $(c_1, \dots, c_m)$ in the kernel of M^t gives rise to the linearizable polynomial

$$p(X) = \sum_{i=0}^{m-1} c_i \cdot X^{2^i}$$

which has $\beta_1, \dots, \beta_m$, and hence the whole space F as roots. By dimension, $p(X)$ must be trivial, meaning that all its coefficients are zero.

B Bilinear Embeddings

Given a field $\mathbb{F}$ and a degree- d extension $\mathbb{E}$. Our goal is to commit to a vector $\mathbf{w} \in \mathbb{F}^{nd}$ so that we can later use it to prove an inner product claim of the form $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = h$ for some $\mathbf{a} \in \mathbb{F}^{nd}$ and $h \in \mathbb{F}$. However, we only have an efficient inner product commitment scheme to commit to vectors $\tilde{\mathbf{w}} \in \mathbb{E}^n$ and open as $\langle \tilde{\mathbf{w}}, \tilde{\mathbf{a}} \rangle_{\mathbb{E}} = \tilde{h}$ for some $\tilde{\mathbf{a}} \in \mathbb{E}^n$ and $\tilde{h} \in \mathbb{E}$.

To do this, we seek a triple of maps (W, A, H) that satisfy:

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(\langle W(\mathbf{w}), A(\mathbf{a}) \rangle_{\mathbb{E}}).$$

B.1 Background

B.1.1 Perfect pairings

In the following let $\mathbb{E}/\mathbb{F}$ be a finite field extension of degree d . If we disregard multiplication in $\mathbb{E}$, then $\mathbb{E}$ is a d -dimensional vector space over $\mathbb{F}$, hence isomorphic to $\mathbb{F}^d$. Let $\mathbf{e}_1, \dots, \mathbf{e}_d$ denote the standard basis of $\mathbb{F}^d$. Recall that a function $\mu : \mathbb{E} \times \mathbb{E} \rightarrow \mathbb{F}$ is a *bilinear form* if it is linear in each argument separately. It is *symmetric* if $\mu(x, y) = \mu(y, x)$ for all $x, y \in \mathbb{E}$. For each $x \in \mathbb{E}$, the partial application in the first argument defines a linear form

$$\mu_x : \mathbb{E} \rightarrow \mathbb{F} \quad y \mapsto \mu(x, y).$$

Thus $\mu_x \in \mathbb{E}^*$ where $\mathbb{E}^* = \text{Hom}_{\mathbb{F}}(\mathbb{E}, \mathbb{F})$ denotes the dual vector space. The bilinear form therefore induces an $\mathbb{F}$ -linear map

$$\Phi_{\mu} : \mathbb{E} \rightarrow \mathbb{E}^* \quad x \mapsto \mu_x.$$

We call μ *nondegenerate*, or equivalently a *perfect pairing*, if Φ_{μ} is an isomorphism. Or equivalently for finite spaces, **Albert:** add a comment that injectivity implies surjectivity when the vector spaces have finite dimension?

$$\mu(x, y) = 0 \text{ for all } y \in \mathbb{E} \implies x = 0.$$

Since μ is a perfect pairing, every basis $(\beta_1, \dots, \beta_d)$ of $\mathbb{E}$ has a unique dual basis $(\beta_1^{\vee}, \dots, \beta_d^{\vee})$ satisfying $\mu(\beta_i, \beta_j^{\vee}) = \delta_{ij}$.

John: It would be good to explain how you would typically derive $\beta_j^{\vee}$

B.1.2 Succinct vectors

Given a field $\mathbb{F}$, a vector $\mathbf{a} \in \mathbb{F}^n$ for $n = 2^k$ is called *succinct* if it admits a tensor decomposition

$$\begin{bmatrix} m_{00} \\ m_{01} \end{bmatrix} \otimes \begin{bmatrix} m_{10} \\ m_{11} \end{bmatrix} \otimes \cdots \otimes \begin{bmatrix} m_{(k-1)0} \\ m_{(k-1)1} \end{bmatrix} \quad (23)$$

where typically $m_{ij} \in \mathbb{F}$. The multilinear extension of $\mathbf{a}$, viewed as a function on $\{0, 1\}^k$, then becomes

$$\tilde{a}(\mathbf{r}) = \prod_{i=0}^{k-1} ((1 - r_i)m_{i0} + r_i m_{i1}). \quad (24)$$

which can be evaluated in $O(k)$ field operations.

More generally, the coefficients m_{ij} need not themselves be field elements. It suffices that they belong to an associative $\mathbb{F}$ -algebra in which every complete product $m_{0b_0}m_{1b_1} \cdots m_{(k-1)b_{k-1}}$ evaluates to an element of $\mathbb{F}$. Consequently, every vector entry and every value of the multilinear extension remains an element of $\mathbb{F}$. As the m_{ij} no longer need to be commutative we must therefore also fix a permutation of the variables $\mathbf{r}$.

Since every finite-dimensional associative algebra admits a matrix representation, we may, without loss of generality, assume that the objects m_{ij} are matrices. In fact, it suffices to take $m_{ij} \in \mathbb{F}^{w_i \times w_{i+1}}$ with boundary dimensions $w_0 = w_k = 1$. We call $\max w_i$ the *width* of the representation, which is known as a *matrix branching program* and is closely related to matrix product states and weighted finite automata.

The canonical example of a succinct vector is the multilinear evaluation vector $a_i = \text{eq}(\mathbf{r}, \text{bits}(i))$ obtained by taking $m_{i0} = 1 - r_i$ and $m_{i1} = r_i$. Another classical example is the geometric progression $(1, x, x^2, \dots)$ obtained by taking $m_{i0} = 1$ and $m_{i1} = x^{2^i}$. An illustrative non-scalar example is the counting vector $(0, 1, 2, \dots)$ represented by

$$m_{0b} = \begin{bmatrix} 2 & b \end{bmatrix} \quad m_{ib} = \begin{bmatrix} 2 & b \\ 0 & 1 \end{bmatrix} \quad m_{(k-1)b} = \begin{bmatrix} b \\ 1 \end{bmatrix}.$$

B.2 Structure

We are now equipped to prove a theorem on the structure of embeddings. Under the assumption that the output map is $\mathbb{F}$ -linear, we show that for $n = 1$ the space of possible embeddings is very structured.

Theorem B.1 (Structure of bilinear embeddings). *Let $W, A : \mathbb{F}^d \rightarrow \mathbb{E}, H : \mathbb{E} \rightarrow \mathbb{F}$ be maps such that for all $\mathbf{w}, \mathbf{a} \in \mathbb{F}^d$,*

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(W(\mathbf{w})A(\mathbf{a})).$$

Suppose that H is $\mathbb{F}$ -linear. Then:

1. W and A are $\mathbb{F}$ -linear isomorphisms,
2. $\mu_H(x, y) = H(xy)$ is a perfect pairing,
3. $(W(\mathbf{e}_1), \dots, W(\mathbf{e}_d))$ and $(A(\mathbf{e}_1), \dots, A(\mathbf{e}_d))$ are dual bases with respect to μ_H .

Conversely, if W and A , and H are $\mathbb{F}$ -linear, μ_H is a perfect pairing and the images of the standard basis under W and A are dual bases with respect to μ_H , then the identity

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(W(\mathbf{w})A(\mathbf{a})).$$

holds for all $\mathbf{w}, \mathbf{a} \in \mathbb{F}^d$.

Proof of the forward direction. First, observe that $\mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) = \langle \mathbf{e}_i, \mathbf{e}_j \rangle_{\mathbb{F}} = \delta_{ij}$. Taking $i = j$ shows H is nonzero. Since H is $\mathbb{F}$ -linear, μ_H is a symmetric bilinear form, we show that it is nondegenerate. Let $x \in \mathbb{E}$ be nonzero. Since multiplication by x is a bijection of $\mathbb{E}$ and H is nonzero, there exists $y \in \mathbb{E}$ such that $\mu_H(x, y) = H(xy) \neq 0$. Thus μ_H is nondegenerate, and hence a perfect pairing.

Next, we show that W and A are bijective. Suppose $W(\mathbf{w}_1) = W(\mathbf{w}_2)$. Then for every $\mathbf{a} \in \mathbb{F}^d$,

$$\langle \mathbf{w}_1 - \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} = \mu_H(W(\mathbf{w}_1) - W(\mathbf{w}_2), A(\mathbf{a})) = 0.$$

By nondegeneracy of the standard inner product on $\mathbb{F}^d$, this implies $\mathbf{w}_1 = \mathbf{w}_2$. Hence W is injective. Since $\mathbb{F}^d$ and $\mathbb{E}$ have the same finite cardinality, W is bijective. The same argument shows that A is bijective.

We now prove that W is $\mathbb{F}$ -linear. For $\mathbf{w}_1, \mathbf{w}_2, \mathbf{a} \in \mathbb{F}^d$, linearity of H gives

$$\begin{aligned} & \mu_H(W(\mathbf{w}_1 + \mathbf{w}_2) - W(\mathbf{w}_1) - W(\mathbf{w}_2), A(\mathbf{a})) \\ &= \langle \mathbf{w}_1 + \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} - \langle \mathbf{w}_1, \mathbf{a} \rangle_{\mathbb{F}} - \langle \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} \\ &= 0. \end{aligned}$$

Since A is surjective, the second argument ranges over all of $\mathbb{E}$. Nondegeneracy of μ_H therefore implies $W(\mathbf{w}_1 + \mathbf{w}_2) = W(\mathbf{w}_1) + W(\mathbf{w}_2)$. Similarly, for $\lambda \in \mathbb{F}$,

$$\begin{aligned} & \mu_H(W(\lambda \mathbf{w}) - \lambda W(\mathbf{w}), A(\mathbf{a})) \\ &= \langle \lambda \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} - \lambda \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} \\ &= 0. \end{aligned}$$

Again using surjectivity of A and nondegeneracy of μ_H , we obtain $W(\lambda \mathbf{w}) = \lambda W(\mathbf{w})$. Hence W is $\mathbb{F}$ -linear. By symmetry, A is also $\mathbb{F}$ -linear. Since they are also bijective, W and A are linear isomorphisms. From this it follows that $(W(\mathbf{e}_1), \dots, W(\mathbf{e}_d))$ and $(A(\mathbf{e}_1), \dots, A(\mathbf{e}_d))$ are bases of $\mathbb{E}$. Finally, from $\mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) = \delta_{ij}$ it follows they are dual with respect to μ_H . $\square$

Proof of the converse direction. Can be directly shown using a basis expansion. Write $\mathbf{w} = \sum_{i=1}^d w_i \mathbf{e}_i$ and $\mathbf{a} = \sum_{j=1}^d a_j \mathbf{e}_j$. Since W and A are $\mathbb{F}$ -linear, $W(\mathbf{w}) = \sum_{i=1}^d w_i W(\mathbf{e}_i)$ and $A(\mathbf{a}) = \sum_{j=1}^d a_j A(\mathbf{e}_j)$. Therefore,

$$\begin{aligned} H(W(\mathbf{w})A(\mathbf{a})) &= \mu_H(W(\mathbf{w}), A(\mathbf{a})) \\ &= \sum_{i,j} w_i a_j \mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) \\ &= \sum_{i,j} w_i a_j \delta_{ij} \\ &= \sum_i w_i a_i \\ &= \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}}. \end{aligned}$$

$\square$

The $n = 1$ case extends naturally to $n > 1$ by doing blockwise packing, i.e. we embed using $W' = I_n \otimes W$ and $A' = I_n \otimes A$ and keep the same H . Let $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_n)$ and $\mathbf{a} = (\mathbf{a}_1, \dots, \mathbf{a}_n)$ then

$$\begin{aligned} H(\langle W'(\mathbf{w}), A'(\mathbf{a}) \rangle_{\mathbb{E}}) &= H\left(\sum_{i=1}^n W(\mathbf{w}_i)A(\mathbf{a}_i)\right) \\ &= \sum_{i=1}^n H(W(\mathbf{w}_i)A(\mathbf{a}_i)) \\ &= \sum_{i=1}^n \langle \mathbf{w}_i, \mathbf{a}_i \rangle_{\mathbb{F}} \\ &= \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}}. \end{aligned}$$

Going forward, we use W and A also for their blockwise extensions when the meaning is clear. We do not consider embeddings that mix coordinates across different blocks of size d , as such embeddings are unlikely to be useful in practice.

We have shown that every $n = 1$ embedding with a linear output map is determined by a choice of a nonzero linear map H and linear isomorphism W . The map W then determines the first basis $\beta_i = W(\mathbf{e}_i)$ and the map A is then forced to map to the μ_H -dual basis.

The linearity assumption on the output map is necessary. Indeed, composing a linear embedding with a multiplicative bijection $g : \mathbb{E} \rightarrow \mathbb{E}$ generally produces an embedding that is nonlinear. Whether every nonlinear embedding arises from such a multiplicative reparameterization is a separate question, which we do not need here.

An important property of bilinear embeddings is that they preserve succinctness if the block size aligns with the tensor decomposition:

Theorem B.2 (Preservation of succinctness). *Given $d = 2^k$, $n = 2^l$ and a $\mathbb{F}$ -linear blockwise map $A = I_n \otimes A_0$ for some isomorphism $A_0 : \mathbb{F}^d \rightarrow \mathbb{E}$, then if $\mathbf{a}$ is succinct then so is $A(\mathbf{a})$. In particular if*

$$\mathbf{a} = \bigotimes_{i=0}^{k+l-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix}$$

where the m_{ij} can be scalars or $w_i \times w_{i+1}$ matrices, then

$$A(\mathbf{a}) = \left(\bigotimes_{i=0}^{l-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix} \right) \cdot (I_{w_l} \otimes A_0) \left(\bigotimes_{i=l}^{l+k-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix} \right).$$

where the second factor is column vector in $\mathbb{E}^{w_l}$ which can be absorbed into the matrices $m_{(l-1)b}$, bringing the representation back into canonical form.

B.3 Constructions

The set of nonzero linear output maps H is $\mathbb{E}^* \setminus \{0\}$ which is isomorphic to $\mathbb{E}^\times$. The set of linear isomorphism W is isomorphic to $\text{GL}_d(\mathbb{F})$, hence the total number of bilinear embeddings is $|\mathbb{E}^\times| \cdot |\text{GL}_d(\mathbb{F})|$. We'll now look at specific choices and their properties.

B.3.1 Trace embedding

Let $q = |\mathbb{F}|$, the q -power Frobenius map is $\varphi : \mathbb{E} \rightarrow \mathbb{E}$ given by $x \mapsto x^q$. It is an $\mathbb{F}$ -linear field automorphism of order d , and its fixed field is precisely $\mathbb{F}$:

$$\mathbb{E}^\varphi = \{x \in \mathbb{E} : \varphi(x) = x\} = \mathbb{F}.$$

The field trace is the $\mathbb{F}$ -linear map $\text{Tr}_{\mathbb{E}/\mathbb{F}} : \mathbb{E} \rightarrow \mathbb{F}$ defined by

$$\text{Tr}_{\mathbb{E}/\mathbb{F}}(x) = \sum_{i=0}^{d-1} \varphi^i(x) = \sum_{i=0}^{d-1} x^{q^i}.$$

To see that this sum is indeed in $\mathbb{F} \subseteq \mathbb{E}$ observe the sum is fixed by φ , since

$$\varphi\left(\sum_{i=0}^{d-1} \varphi^i(x)\right) = \sum_{i=1}^d \varphi^i(x) = \sum_{i=0}^{d-1} \varphi^i(x),$$

where $\varphi^d(x) = \varphi^0(x)$. Hence the trace lies in the fixed field $\mathbb{F}$.

This makes $\text{Tr}_{\mathbb{E}/\mathbb{F}}$ a natural choice of output map H . More generally, every $\mathbb{F}$ -linear map $H : \mathbb{E} \rightarrow \mathbb{F}$ is uniquely of the form $H(x) = \text{Tr}_{\mathbb{E}/\mathbb{F}}(\alpha x)$ for some $\alpha \in \mathbb{E}$. Consequently, the nonzero output maps are parameterized by $\mathbb{E}^\times$.

The Galois orbit embedding chooses the basis $\beta_i = \varphi^i(\gamma)$ which has dual $\beta_i^\vee = \varphi^i(\gamma^\vee)$.

Ulrich: This sounds to me very much related to Galois Frobenius on an inner product with a general covector. You would end up with the multi-claim of the inner products of the packed poly with the Galois orbit of the covector. This is how I view it: Given $\mathbf{p}_1, \dots, \mathbf{p}_d \in \mathbb{F}^n$, and $\mathbf{q}_1, \dots, \mathbf{q}_d \in \mathbb{F}^n$, we are interested in

$$\sum_{k=1}^d \langle \mathbf{p}_k, \mathbf{q}_k \rangle,$$

right?

The packing map sends the coeff vectors $\mathbf{p}_1, \dots, \mathbf{p}_d$ to $\mathbf{p} = \beta_1 \cdot \mathbf{p}_1 + \dots + \beta_d \cdot \mathbf{p}_d$, where $\beta_1, \dots, \beta_d$ is an $\mathbb{F}$ -basis of $\mathbb{E}$. Choose the element a_k in $\mathbb{E}$ which extracts the k -th coordinate

$$\mathbf{p}_k = \Phi_k(\mathbf{p}) = \text{Tr}_{\mathbb{E}/\mathbb{F}}(a_k \cdot \mathbf{p}).$$

Then

$$\begin{aligned} \langle \mathbf{p}_k, \mathbf{q}_k \rangle &= \left\langle \sum_{i=0}^{d-1} \varphi^i(a_k \cdot \mathbf{p}), \mathbf{q}_k \right\rangle \\ &= \sum_{i=0}^{d-1} \varphi^i(a_k) \cdot \varphi^i(\langle \mathbf{p}, \mathbf{q}_k \rangle), \end{aligned}$$

since $\mathbf{q}_k$ is over $\mathbb{F}$. This reduces our inner product claim to that of the packed poly $\mathbf{p}$ with the unpacked polys $\mathbf{q}_1, \dots, \mathbf{q}_k$.

But does that match our use case? We are rather interested in $\langle \mathbf{p}_k, \mathbf{v} \rangle$ for some $\mathbf{v} \in \mathbb{E}^n$. In this case,

$$\langle \mathbf{p}_k, \mathbf{v} \rangle = \left\langle \sum_{i=0}^{d-1} \varphi^i(a_k \cdot \mathbf{p}), \mathbf{q}_k \right\rangle$$

$$= \sum_{i=0}^d \varphi^i(a_k) \cdot \varphi^i(\langle \mathbf{p}, \varphi^{-i}(\mathbf{v}) \rangle),$$

and we end up with inner products of the packed poly with the Galois orbit of $\mathbf{v}$.

B.3.2 Coefficient projection

In the coefficient projection we assume a concrete representation of $\mathbb{E}$ as $\mathbb{F}[X]/(f(X))$ for some monic degree d polynomial f irreducible over $\mathbb{F}$. We assume f is chosen for implementation efficiency: it is sparse with mostly low order terms. Further we represent members of $\mathbb{E}$ in the monomial basis $(1, X, X^2, \dots, X^{d-1})$. We can then represent W and A by invertible matrices in $\mathbb{F}^{d \times d}$ and H by a vector in $\mathbb{F}^d$. For implementation efficiency we seek an embedding with sparse W and A . It turns out taking the most efficient choices for H and W also gives efficient A :

Theorem B.3. *Write $f(X) = f_d X^d + f_{d-1} X^{d-1} + \dots + f_1 X + f_0$ so that $f_d = 1$ and $f_0 \neq 0$. Take $W = I_d$ and $H = (1, 0, \dots, 0)$. Then the dual-basis embedding A is given by*

$$A_{ij} = \begin{cases} 1, & i = j = 0, \\ -f_{i+j}/f_0, & i, j \geq 1 \text{ and } i + j \leq d, \\ 0, & \text{otherwise.} \end{cases}$$

In particular, A is a bordered upper anti-triangular Hankel matrix. For $d = 5$ it looks like

$$A = \frac{1}{f_0} \begin{pmatrix} f_0 & 0 & 0 & 0 & 0 \\ 0 & -f_2 & -f_3 & -f_4 & -1 \\ 0 & -f_3 & -f_4 & -1 & 0 \\ 0 & -f_4 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 \end{pmatrix}.$$

Notably f_1 does not appear and for $k > 1$ the f_k coefficient appears $k - 1$ times. This perfectly matches the usual implementation preference for sparse reduction polynomials with mostly low-order terms.

For the 127-bit field $\mathbb{F}_2[X]/(X^{127} + X + 1)$ the A map is simply a reversal of the $i > 0$ coordinates. For the GHASH field $\mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$ the map additionally needs seven XOR corrections.

B.4 Extension openings

Assume now that we wish to open a base-field witness $\mathbf{w} \in \mathbb{F}^{nd}$ against extension-field weights $\mathbf{a} \in \mathbb{E}^{nd}$, proving the claim $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$ for some $h \in \mathbb{E}$. We present a one-round batching protocol and a $\log d$ -round folding protocol.

B.4.1 Batching protocol

Given a bilinear embedding (W, A, H) and a basis $(\beta_1, \dots, \beta_d)$ for $\mathbb{E}/\mathbb{F}$, we reduce the extension-field claim by decomposing into basis coordinates and batching the resulting claims with a random linear combination.

Construction B.4 (Batching protocol for extension opening). Given the claim $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$, define $\mathbf{a}_i \in \mathbb{F}^{nd}$ such that

$$\mathbf{a} = \sum_{i=1}^d \mathbf{a}_i \beta_i.$$

The parties proceed as follows.

1. The prover sends $h_i = \langle W(\mathbf{w}), A(\mathbf{a}_i) \rangle_{\mathbb{E}}$ for $i = 1, \dots, d$.
2. The verifier checks $h = \sum_{i=1}^d H(h_i) \beta_i$.
3. The verifier samples an ϵ -zero-evader $\boldsymbol{\rho} \leftarrow \$_{\epsilon} \mathbb{E}^d$.
4. The parties reduce the claim to $\langle W(\mathbf{w}), \mathbf{a}' \rangle_{\mathbb{E}} = h'$ where

$$h' = \sum_{i=1}^d \rho_i h_i, \quad \mathbf{a}' = \sum_{i=1}^d \rho_i A(\mathbf{a}_i).$$

This interactive oracle reduction is always complete, has round-by-round soundness error ϵ , and proof size $d \log |\mathbb{E}| = d^2 \log |\mathbb{F}|$.

Theorem B.5 (Succinctness of coordinate decomposition). *Given a succinct vector $\mathbf{a} \in \mathbb{E}^{nd}$ for $d = 2^k$ and a basis (β_i) of $\mathbb{E}/\mathbb{F}$ then the β -coordinate decomposed vectors are also succinct. In particular if*

$$\mathbf{a} = \bigotimes_{i=0}^{k+l-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix}$$

where the m_{ij} can be scalars or $w_i \times w_{i+1}$ matrices, then

$$\mathbf{a}_i = \mathbf{u}_i \cdot \left(\bigotimes_{j=0}^{k+l-1} \begin{bmatrix} \Phi(m_{j0}) \\ \Phi(m_{j1}) \end{bmatrix} \right) \cdot \mathbf{v}$$

where $\Phi : \mathbb{E} \rightarrow \mathbb{F}^{d \times d}$ is the map that represents an $\mathbb{E}$ -element m by the matrix representing multiplication by m in basis (β_i) , $\mathbf{u}_i$ is the one-hot vector with $u_{ij} = \delta_{ij}$, and $\mathbf{v}$ is the base (β_i) representation of $1 \in \mathbb{E}$. Φ is applied elementwise to $w_i \times w_{i+1}$ matrices over $\mathbb{E}$ to produce $dw_i \times dw_{i+1}$ block matrices over $\mathbb{F}$. Absorbing the outer vectors in the matrices brings the representation back into canonical form, with the width increased by at most a factor d .

For practical implementations it makes sense to evaluate the branching program over $\mathbb{E}$ directly and then extract the coordinates in the basis (β_i) . This is equivalent to omitting the output selector $\mathbf{t}_i$ to keep the output dimension $w_0 = d$.

If $A = I_n \otimes A_0$ is blockwise we can also apply theorem ?? and compose it with the combination weights $\boldsymbol{\rho} \in \mathbb{E}^d$ to get a succinct representation for $\mathbf{a}'$:

$$\mathbf{a}' = \boldsymbol{\rho} \cdot \left(\bigotimes_{i=0}^{l-1} \begin{bmatrix} \Phi(m_{i0}) \\ \Phi(m_{i1}) \end{bmatrix} \right) \cdot (I_{dw_l} \otimes A_0) \left(\bigotimes_{i=l}^{l+k-1} \begin{bmatrix} \Phi(m_{i0}) \\ \Phi(m_{i1}) \end{bmatrix} \right) \cdot \mathbf{v}. \quad (25)$$

The map Φ is an algebra embedding, so replacing each scalar $m_{ij} \in \mathbb{E}$ by $\Phi(m_{ij})$ leaves the branching-program computation unchanged, except that the values are now represented in the basis (β_i) representation of $\mathbb{E}$. The only operations outside the embedded $\mathbb{E}$ computation are the blockwise embedding A_0 and the random linear combination ρ . Thus, apart from the applications of A_0 and ρ , evaluating $\tilde{\mathbf{a}}'$ is the same computation as evaluating $\tilde{\mathbf{a}}$, performed in the regular representation of $\mathbb{E}$. **Remco: Something is off with the complexity. Should really check with a Sage implementation. Probably needs a factor d .**

The batch protocol is quadratic in d in terms of $\mathbb{F}$ -element operations and proof size. This is acceptable when d is small, but binary fields where $d = 128$ the communication would be 2,048 bytes. Avoiding this quadratic dependence on d appears to require further rounds of interaction and more sophisticated decomposition.

B.4.2 Folding protocol

In the folding protocol we interactively reduce the claim $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$ through a sequence of embeddings W_i where at round i the claim is

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle_{\mathbb{E}} = h_i \quad W_i : \mathbb{F}^{nd} \rightarrow \mathbb{E}^{nd_i} \quad A_i : \mathbb{E}^{nd} \rightarrow \mathbb{E}^{nd_i}$$

Here $d = d_0 \geq \dots \geq d_k = 1$ is a series of folding dimensions. We start with W_0 and A_0 the identity and $h_0 = h$ so that the initial claim is indeed $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$. We also stipulate that the embeddings are blockwise such that $W_i = I_n \otimes W'_i$ and $A_i = I_n \otimes A'_i$. For simplicity we will only consider the $n = 1$ case from here on as $n > 1$ follows by blockwise repetition.

The final map is $W_k : \mathbb{F}^d \rightarrow \mathbb{E}$ which is an F -linear isomorphism. Let (β_i) be the implied basis such that

$$W_k(\mathbf{x}) = \sum_{i=1}^d x_i \beta_i.$$

Working backwards from the final embedding we create a sequence

$$R_i \circ W_{i+1} = W_i$$

where each map R_i has only $\left\lceil \frac{d_i}{d_{i+1}} \right\rceil$ Frobenius components.

Every $\mathbb{F}$ -linear map $T : \mathbb{E}^m \rightarrow \mathbb{E}^n$ can be viewed as being built from $\mathbb{E}$ -linear maps and $\mathbb{F}$ -linear scalar operations on $\mathbb{E}$. To recursively fold such maps, the scalar operations must be compatible with arbitrary $\mathbb{E}$ -linear coordinate transformations. Concretely, we require an intertwining

$$\beta(R\mathbf{x}) = \alpha(R)\beta(\mathbf{x}),$$

where α acts entrywise on R . This property singles out semilinear scalar operators. Over finite fields, these are precisely the Frobenius maps σ^i up to scalar multiples, and the Frobenius powers therefore provide the canonical scalar decomposition.

Theorem B.6 (Frobenius decomposition). *Let $\mathbb{E}/\mathbb{F}$ be a finite extension of degree d , and let $\sigma : \mathbb{E} \rightarrow \mathbb{E}$ be the Frobenius map $x \mapsto x^{|\mathbb{F}|}$. Then every $\mathbb{F}$ -linear map $T : \mathbb{E}^m \rightarrow \mathbb{E}^n$ admits a unique decomposition*

$$T(\mathbf{x}) = \sum_{i=0}^{d-1} M_i \cdot (\mathbf{I}_m \otimes \sigma^i)(\mathbf{x}), \quad M_i \in \mathbb{E}^{n \times m}.$$

We define $\text{supp}_\sigma(T) := \{i : M_i \neq 0\}$.

Single folding round. Assume the current claim is

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle_{\mathbb{E}} = h_i.$$

Let

$$R_i : \mathbb{E}^{nd_{i+1}} \rightarrow \mathbb{E}^{nd_i}$$

be a fixed $\mathbb{F}$ -linear map satisfying

$$R_i \circ W_{i+1} = W_i.$$

By the Frobenius decomposition there exist unique matrices

$$M_{i,0}, \dots, M_{i,d-1} \in \mathbb{E}^{nd_i \times nd_{i+1}}$$

such that

$$R_i = \sum_{j=0}^{d-1} M_{i,j} \circ (\mathbf{I}_{nd_{i+1}} \otimes \sigma^j).$$

For every j , define the $\mathbb{F}$ -linear map

$$C_{i,j} = (\mathbf{I}_{nd_{i+1}} \otimes \sigma^{-j}) \circ M_{i,j}^\top.$$

Then, for every $\mathbf{x} \in \mathbb{E}^{nd_{i+1}}$ and $\mathbf{b} \in \mathbb{E}^{nd_i}$,

$$\langle R_i(\mathbf{x}), \mathbf{b} \rangle_{\mathbb{E}} = \sum_{j=0}^{d-1} \sigma^j(\langle \mathbf{x}, C_{i,j}(\mathbf{b}) \rangle_{\mathbb{E}}).$$

Applying this identity to

$$\mathbf{x} = W_{i+1}(\mathbf{w}), \quad \mathbf{b} = A_i(\mathbf{a}),$$

gives

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle_{\mathbb{E}} = \sum_{j=0}^{d-1} \sigma^j(\langle W_{i+1}(\mathbf{w}), C_{i,j}(A_i(\mathbf{a})) \rangle_{\mathbb{E}}).$$

The prover sends the values

$$g_{i,j} = \langle W_{i+1}(\mathbf{w}), C_{i,j}(A_i(\mathbf{a})) \rangle_{\mathbb{E}}, \quad j = 0, \dots, d-1.$$

The verifier checks

$$h_i = \sum_{j=0}^{d-1} \sigma^j(g_{i,j}),$$

samples a random challenge $\rho_i \in \mathbb{E}$, and defines

$$A_{i+1} = \left(\sum_{j=0}^{d-1} \rho_i^j C_{i,j} \right) \circ A_i.$$

Finally, letting

$$h_{i+1} = \sum_{j=0}^{d-1} \rho_i^j g_{i,j},$$

the next claim is

$$\langle W_{i+1}(\mathbf{w}), A_{i+1}(\mathbf{a}) \rangle_{\mathbb{E}} = h_{i+1}.$$

We therefore seek an interactive reduction from a mixed claim to a packed claim

$$\begin{array}{ccc} \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h & \mathbf{w} \in \mathbb{F}^{nd} & \mathbf{a} \in \mathbb{E}^{nd} \\ \Downarrow & & \\ \langle W(\mathbf{w}), \mathbf{a}' \rangle_{\mathbb{E}} = h' & W(\mathbf{w}) \in \mathbb{E}^n & \mathbf{a}' \in \mathbb{E}^n \end{array}$$

where $W : \mathbb{F}^{nd} \rightarrow \mathbb{E}^n$ is a fixed $\mathbb{F}$ -linear embedding and $\mathbf{a}' \in \mathbb{E}^n$, $h' \in \mathbb{E}$ are determined by $\mathbf{a}$, h and the transcript. Ideally $\mathbf{a} \mapsto \mathbf{a}'$ should also preserve useful properties like efficient computation of its multilinear extension.

Above we decomposed $\mathbf{a}$ with respect to a basis of $\mathbb{E}/\mathbb{F}$. Here we instead decompose the induced $\mathbb{F}$ -linear functional on packed witnesses. Define

$$T_{\mathbf{a}} : \mathbb{E}^n \rightarrow \mathbb{E} \quad \mathbf{x} \mapsto \langle W^{-1}(\mathbf{x}), \mathbf{a} \rangle_{\mathbb{E}}.$$

This is an $\mathbb{F}$ -linear functional on $\mathbb{E}^n$, but is not generally $\mathbb{E}$ -linear. We'll first prove a representation lemma for $\mathbb{F}$ -linear $\mathbb{E} \rightarrow \mathbb{E}$ maps and then a decomposition theorem.

Theorem B.7. *Let $\mathbb{F} \subseteq \mathbb{E}$ be finite fields with $q = |\mathbb{F}|$ and $[\mathbb{E} : \mathbb{F}] = d$, then every $\mathbb{F}$ -linear map $T : \mathbb{E} \rightarrow \mathbb{E}$ is uniquely represented by a q -linearized polynomial*

$$P(X) = \sum_{j=0}^{d-1} c_j X^{q^j} \quad c_j \in \mathbb{E}.$$

Proof. By interpolation every map $T : \mathbb{E} \rightarrow \mathbb{E}$ can be represented uniquely by a polynomial $P \in \mathbb{E}[X]$ of degree less than $|\mathbb{E}| = q^d$

$$P(X) = \sum_{j=0}^{q^d-1} a_j X^j \quad a_j \in \mathbb{E}.$$

From $\mathbb{F}$ -linearity follows $P(0) = 0$ and hence $a_0 = 0$, further we have $P(X + Y) = P(X) + P(Y)$ and hence

$$\begin{aligned} 0 &= P(X + Y) - P(X) - P(Y) \\ &= \sum_{j=1}^{q^d-1} a_j (X + Y)^j - \sum_{j=1}^{q^d-1} a_j (X^j + Y^j) \\ &= \sum_{j=1}^{q^d-1} a_j \sum_{k=0}^j \binom{j}{k} X^k Y^{j-k} - \sum_{j=1}^{q^d-1} a_j (X^j + Y^j) \\ &= \sum_{j=1}^{q^d-1} a_j \sum_{k=1}^{j-1} \binom{j}{k} X^k Y^{j-k} \end{aligned}$$

Write $q = p^r$ with p the characteristic of $\mathbb{F}$. For every j with $a_j \neq 0$ we must have $\binom{j}{k} = 0 \pmod{p}$ for all $0 < k < j$, by Lucas's theorem this holds iff $j = p^s$ for some s . Hence we can reindex P keeping only the non-zero coefficients

$$P(X) = \sum_{j=0}^{rd-1} b_j X^{p^j} \quad b_j \in \mathbb{E}.$$

From $\mathbb{F}$ -linearity also follows $P(\lambda x) = \lambda P(x)$ for all $x \in \mathbb{E}$ and $\lambda \in \mathbb{F}$. For every $b_j \neq 0$ this requires $\lambda^{p^j} = \lambda$ for every $\lambda \in \mathbb{F}$. This holds exactly when $j = rs$ for some s . Reindexing P again we get our claimed result

$$P(X) = \sum_{j=0}^{d-1} c_j X^{q^j} \quad c_j \in \mathbb{E}.$$

□

Theorem B.8 (Frobenius decomposition). *Let $\mathbb{F} \subseteq \mathbb{K} \subseteq \mathbb{E}$ be finite fields with $[\mathbb{K} : \mathbb{F}] = m$, and let $\sigma(x) = x^{|\mathbb{F}|}$ denote the $\mathbb{F}$ -Frobenius automorphism. Then every $\mathbb{F}$ -linear map $T : \mathbb{E} \rightarrow \mathbb{E}$ admits a unique decomposition*

$$T(x) = \sum_{i=0}^{m-1} \sigma^i \circ T_i$$

where each T_i is $\mathbb{K}$ -linear.

Proof. Write $q = |\mathbb{F}|$ and $d = [\mathbb{E} : \mathbb{F}]$, by the preceding theorem T has a unique representation

$$T(X) = \sum_{j=0}^{d-1} c_j X^{q^j} \quad c_j \in \mathbb{E}.$$

Since $\mathbb{K} \subseteq \mathbb{E}$, we have $m \mid d$. Write each index uniquely as $j = ms + i$, with $0 \leq i < m$ and $0 \leq s < d/m$. Grouping the terms by their residue modulo m gives

$$T(X) = \sum_{i=0}^{m-1} \sum_{s=0}^{d/m-1} c_{ms+i} X^{q^{ms+i}}.$$

For each residue class i , define

$$T_i(X) = \sum_{s=0}^{d/m-1} c_{ms+i}^{q^{-i}} X^{q^{ms}},$$

where $z \mapsto z^{q^{-i}}$ denotes the inverse of the automorphism $z \mapsto z^{q^i}$ of $\mathbb{E}$. Each T_i is now a q^m -linearized polynomial hence applying the preceding theorem to the extension $\mathbb{K} \subseteq \mathbb{E}$, each T_i is $\mathbb{K}$ -linear. Then

$$(\sigma^i \circ T_i)(X) = T_i(X)^{q^i} = \sum_{s=0}^{d/m-1} c_{ms+i} X^{q^{ms+i}}.$$

Therefore T represented as

$$T = \sum_{i=0}^{m-1} \sigma^i \circ T_i.$$

Uniqueness follows immediately from the uniqueness of the q -linear polynomial representation. $\square$

The decomposition theorem extends coordinatewise to maps $\mathbb{E}^n \rightarrow \mathbb{E}$. In particular, the functional $T_{\mathbf{a}}(\mathbf{x}) = \langle W^{-1}(\mathbf{x}), \mathbf{a} \rangle_{\mathbb{E}}$ admits a unique decomposition

$$T_{\mathbf{a}} = \sum_{i=0}^{m-1} \sigma^i \circ T_i, \tag{26}$$

where each $T_i : \mathbb{E}^n \rightarrow \mathbb{E}$ is $\mathbb{K}$ -linear. Thus the initial claim may be written as

$$h = T_{\mathbf{a}}(W(\mathbf{w})) = \sum_{i=0}^{m-1} \sigma^i(T_i(W(\mathbf{w}))).$$

Construction B.9 (Frobenius fold). Given the claim $T_{\mathbf{a}}(W(\mathbf{w})) = h$, the parties proceed as follows.

1. The prover sends $v_i = T_i(W(\mathbf{w}))$ for $i = 1, \dots, m-1$.
2. The verifier derives $v_0 = h - \sum_{i=1}^{m-1} \sigma^i(v_i)$.
3. The verifier samples independent challenges $\rho_1, \dots, \rho_{m-1} \leftarrow \$ \mathbb{E}$.
4. The parties reduce to the claim $T'(W(\mathbf{w})) = h'$, where

$$T' = T_0 + \sum_{i=1}^{m-1} \rho_i T_i \quad \text{and} \quad h' = v_0 + \sum_{i=1}^{m-1} \rho_i v_i.$$

Construction B.10 (Folding protocol for extension opening). Choose a tower of intermediate fields $\mathbb{F} = \mathbb{K}_0 \subseteq \mathbb{K}_1 \subseteq \dots \subseteq \mathbb{K}_t = \mathbb{E}$, and write $m_i = [\mathbb{K}_{i+1} : \mathbb{K}_i]$. Initially, $T_{\mathbf{a}}$ is $\mathbb{K}_0$ -linear. At round i , apply the Frobenius fold with $\mathbb{F} = \mathbb{K}_i$, and $\mathbb{K} = \mathbb{K}_{i+1}$. This reduces the current $\mathbb{K}_i$ -linear functional to a $\mathbb{K}_{i+1}$ -linear functional.

After t rounds, the residual functional $T' : \mathbb{E}^n \rightarrow \mathbb{E}$ is $\mathbb{E}$ -linear. Consequently, there exists $\mathbf{a}' \in \mathbb{E}^n$ such that $T'(\mathbf{x}) = \langle \mathbf{x}, \mathbf{a}' \rangle_{\mathbb{E}}$. The final claim is therefore $\langle W(\mathbf{w}), \mathbf{a}' \rangle_{\mathbb{E}} = h'$, as required.

Each round requires sending $m_i - 1$ elements of $\mathbb{E}$. For a binary tower where $m_i = 2$, $\mathbb{F} = \mathbb{F}_2$, and $\mathbb{E} = \mathbb{F}_{2^{128}}$ we have 7 rounds of one $\mathbb{E}$ element message each, for a total of 112 bytes. Compare to the non-interactive protocol this reduces the communication complexity from $O(d^2)$ to $O(d \log d)$ elements of $\mathbb{F}_2$.

Towerless folding. Say $\mathbb{F} \subseteq \mathbb{E}$ admits no tower of subfields, for example $\mathbb{E} = \mathbb{F}_{2^{127}}$. Take again $d = [\mathbb{E} : \mathbb{F}]$ and decompose $T_{\mathbf{a}}$ as

$$T_{\mathbf{a}} = \sum_{i=0}^{d-1} \sigma^i \circ T_i,$$

We can still recursively fold using the protocol above with $m = 2$ by grouping even and odd exponents of σ^i (zero-padding as necessary), as there is no actual dependence on the subfield.

This folding progressively halves d (rounding up) in the $\mathbb{E}^{nd}$ inner product. This induces a sequence of perfect pairings

$$(\mathbb{F}^{nd}, \mathbb{E}^{nd}) \rightsquigarrow (\mathbb{E}^{nd/m}, \mathbb{E}^{nd/m}) \rightsquigarrow (\mathbb{K}_i^{nd/m}, \mathbb{E}^{nd/m})$$

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle = \langle \mathbf{w}, \mathbf{a} \rangle.$$

The final result is a transcript dependent map $A : \mathbb{E}^{dn} \rightarrow \mathbb{E}^n$. If W is blockwise with $I_n \otimes W_0$ for some $W_0 : \mathbb{F}^d \rightarrow \mathbb{E}$ then so is A with $A_0 : \mathbb{E}^d \rightarrow \mathbb{E}$.

C GKR protocol for multiple grand products